



ML Competency Center

For Internal Use Only

Resolution Life

Machine Learning Competency Center Whitepaper

AWS Professional Services
June 2022

TABLE OF CONTENTS

Context	3
EXECUTIVE SUMMARY.....	4
Expected Outcomes.....	4
BACKGROUND	5
ML Competency Center Delivery Approach	5
Resolution Life MLCC Technical ASSESSMENT	7
Current State	7
Target State	8
Assessment Results	8
Resolution Life MLCC IMPLEMENTATION RECOMMENDATION	9
Success Metrics and Benefits	10
DETAILED TACTICAL ML USE CASES	10
MLCC Tactical Use Case #10: Data Ingestion Pipeline	11
MLCC Tactical Use Case #11: ML Data Engine	14
MLCC Tactical Use Case #12: ML Experimentation	17
MLCC Tactical Use Case #13: ML Model Management	24
MLCC Tactical Use Case #14: Evaluation and Performance.....	30
MLCC Tactical Use Case #15: ML Implementation Pipeline	36
MLCC Tactical Use Case #16: ML Inference Delivery.....	44
PRIORITIZATION.....	49
RECOMMENDED NEXT STEPS	50
OPTIONS FOR SUPPORT	51
ADDITIONAL NOTES & ASSUMPTIONS	51
Participants.....	53

ML COMPETENCY CENTER ASSESSMENT

RESOLUTION LIFE

6/17/2022

CONTEXT

AWS conducted a Machine Learning Competency Center (MLCC) Assessment with the Resolution Life team on 6/14/2022. The MLCC Assessment focused on key areas related to enabling machine learning (ML) at enterprise scale within Resolution Life. During the assessment we identified strengths and gaps across the Technical area at Resolution Life, all with a lens towards helping Resolution Life accelerate the deployment and realization of business value through broader ML endeavors. The output of the process is included in this whitepaper, creating a roadmap for Resolution Life to extend areas of strength, address critical gaps, and ultimately develop an ML-first enterprise organization.

AWS recommendations including a series of tactical MLCC use cases to realize the above goals:

- Data Ingest Pipeline
- ML Data Engine
- ML Experimentation
- ML Model Management
- Evaluation and Performance
- ML Implementation Pipeline
- ML Inference Delivery

In seeking to strengthen their ML platform and operations, the Resolution Life team requires implementing an overall MLCC approach that delivers best practices in creating data and ML pipelines and processes to streamline model development and deployment. The vision for the Resolution Life MLCC is to realize:

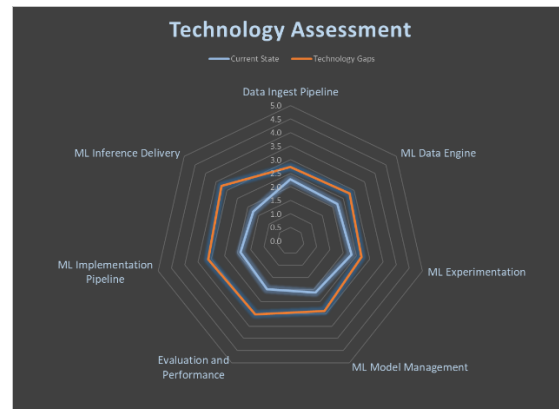
- **Visibility across projects** – deliver insights into every ML project, capturing learnings and managing assets, and sharing knowledge gained throughout the organization
- **Common philosophy, approach, and discipline around data science and production** – at a tooling and methods level integrate across projects, enable DevOps functionality, and shatter the cultural belief that ML experimentation and production are separate functions
- **Instill best practices** – Enable the documentation, collection, and use of best practices, lessons learned, and knowledge developed across the entire Resolution Life organization
- **Flexibility and standardization** – Maintain flexibility in ML model development activities, while establishing standardization in ML engineering, ML operations, and model deployment

EXECUTIVE SUMMARY

Resolution Life Holdings Inc. offers life insurance products and services. The Company focuses on policyholder services and delivering sustainable financial results within a risk framework.

During the Machine Learning Technical assessment, the following areas have been identified as needing improvement:

- **Data Ingest Pipeline**
- **ML Model Management**
- **ML Experimentation**
- **ML Inference Delivery**
- **ML Data Engine**
- **ML Implementation Pipeline**
- **Evaluation and Performance**



EXPECTED OUTCOMES

Resolution Life's expected outcomes of the MLCC engagement include:

- Enabling tools and processes to create an ML platform to be used across teams
- Ability to build, train, and deploy ML models for internal and end-user customers with ease
- Enabling ML at scale
- ML governance and an ML operational model that internal resources can follow
- Cost optimization opportunities.

Given Resolution Life's organizational requirements, AWS will develop a program which incorporates tactical use cases across these pillars, which will enable learnings and advancement on all fronts, enabling Resolution Life to ultimately operate as an ML-First company independently and efficiently.

In this initial assessment, due to business priorities and needs, Resolution Life focused on the Technology pillar of the MLCC. Future operational and resourcing assessments will be scheduled as needed.

BACKGROUND

During the MLCC Assessments, the AWS team discussed Resolution Life's end goals in scaling internal ML capabilities into the organization, identifying the pillar priorities and a number of tactical use cases, across multiple business lines, along with data availability, business processes, and supporting knowledge impactful to the use cases discussed.

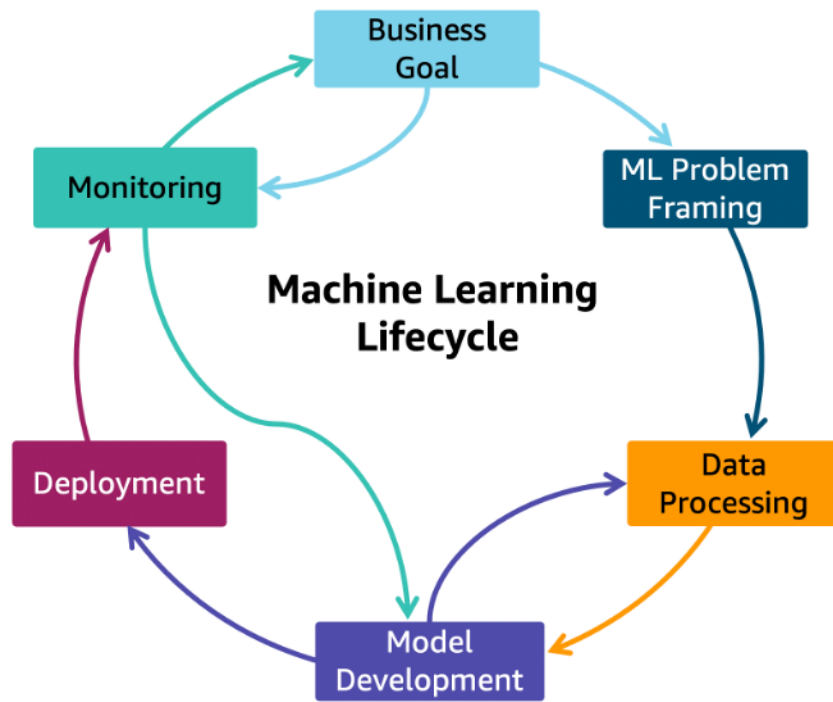
Resolution Life is looking for an assessment of their current capabilities, a gap analysis comparing its features with best practices in the industry, as well as recommendations on how to close those gaps.

ML COMPETENCY CENTER DELIVERY APPROACH

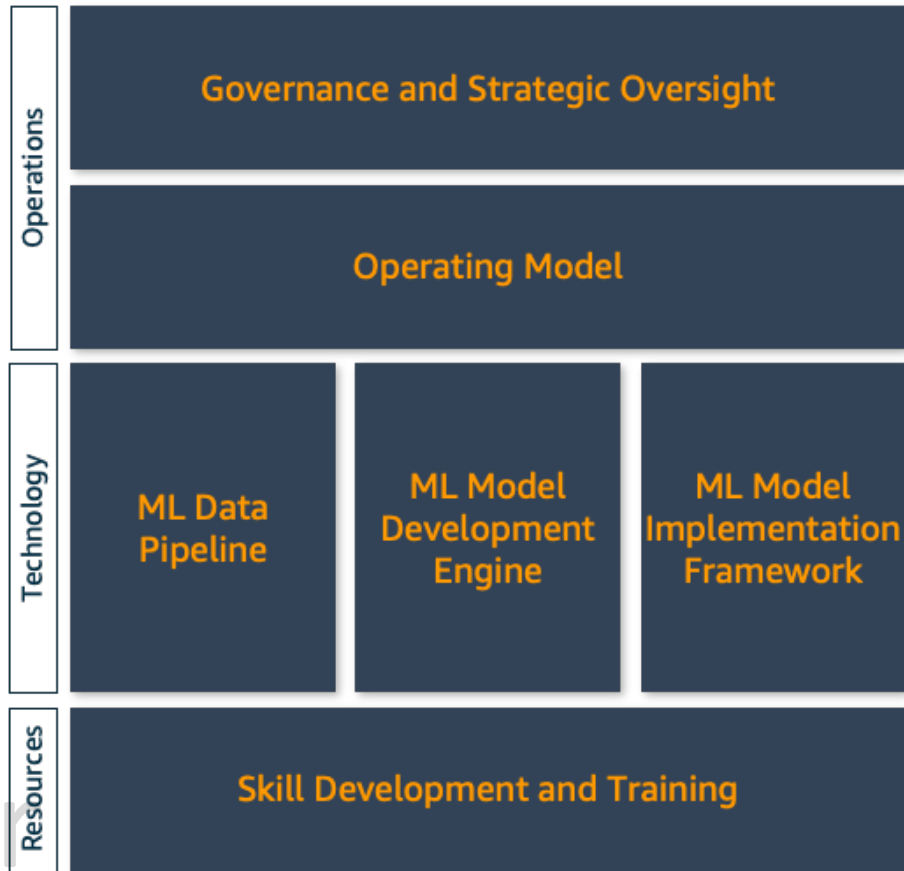
The ML Competency Center (MLCC) considers all aspects of the standard ML lifecycle which comprises the following defined phases:

- **Business goal identification** - An organization considering ML should have a clear idea of the problem, and the business value to be gained by solving that problem. You must be able to measure business value against specific business objectives and success criteria.
- **ML problem framing** - In this phase, the business problem is framed as a machine learning problem: what is observed and what should be predicted (known as a label or target variable). Determining what to predict and how performance and error metrics must be optimized is a key step in this phase
- **Data processing** (data collection, data preprocessing, feature engineering) - Training an accurate ML model requires data processing to convert data into a usable format. Data processing steps include collecting data, preparing data, and feature engineering that is the process of creating, transforming, extracting, and selecting variables from data.
- **Model development** (training, tuning, evaluation) - - Model development consists of model building, training, tuning, and evaluation. Model building includes creating a CI/CD pipeline that automates the build, train and release to staging and production environments.
- **Model deployment** (inference, prediction) - After a model is trained, tuned, evaluated and validated, you can deploy the model into production. You can then make predictions and inferences against the model.
- **Model monitoring** - Model monitoring system ensures your model is maintaining a desired level of performance through early detection and mitigation.

The phases of the ML lifecycle are not necessarily sequential in nature and can have feedback loops, a few of which are illustrated below, to interrupt the cycle across the lifecycle phases.



The ML Competency Center (MLCC) assessment is designed to around three core pillars, Operations, Technology and Resources, and analyses the ML capability as illustrated below, to guide the embedding of AWS best practices for developing an enterprise-wide ML capability that can be used across departments and business lines.



For Resolution Life, we focused on the Technology pillar assessment.

The tailoring of the MLCC delivery approach to Resolution Life’s unique environment, current capability strengths, and operational realities the organization is achieved through the targeted MLCC building block projects targeted to address Resolution Life’s enterprise ML priorities, reducing key pain points and enabling enterprise capabilities and guardrails, as described in detail below.

RESOLUTION LIFE MLCC TECHNICAL ASSESSMENT

Through one assessment for the Technical Pillar/ML Ops only, a number of strengths, areas of enhancement, and gaps were identified in Resolution Life’s ML organization.

CURRENT STATE

Resolution Life faces a variety of challenges in implementing ML capabilities currently across the organization, including:

Technology:

- Data management and separation of sensitive data (PII) needs to be improved.
- ML experimentation and production is currently all done manually and on an ad-hoc basis, will need to be automated.
- One model in production currently and need a robust process to deploy multiple models in production in an automated manner following best practices.
- Management of model performance/drift is not currently captured in production and will need to be in target state.
- Model retraining and continuous improvement in production is not currently implemented and will need to be in target state.
- Code as infrastructure and CI/CD will need to be implemented with best practices.

TARGET STATE

Resolution Life's vision for the optimal ML operational end state is designed to address these gaps and areas of enhancement, and includes the following:

Technology:

- Will have defined data dictionaries, data security management (encryption), and management of sensitive data in place.
- Will have an end-to-end ML platform built with CI/CD pipeline for model experimentation and development, model inference in production, and model maintenance in the form of model retraining and continuous tuning.
- Will have a deployment template with a wide array of modeling frameworks to choose from (e.g., TensorFlow, etc.), algorithms (e.g., Gradient Boosting Machines, neural networks), and interfaces (e.g., Keras, Gluon)
- Have the ability to do real-time model comparisons to assess model performance

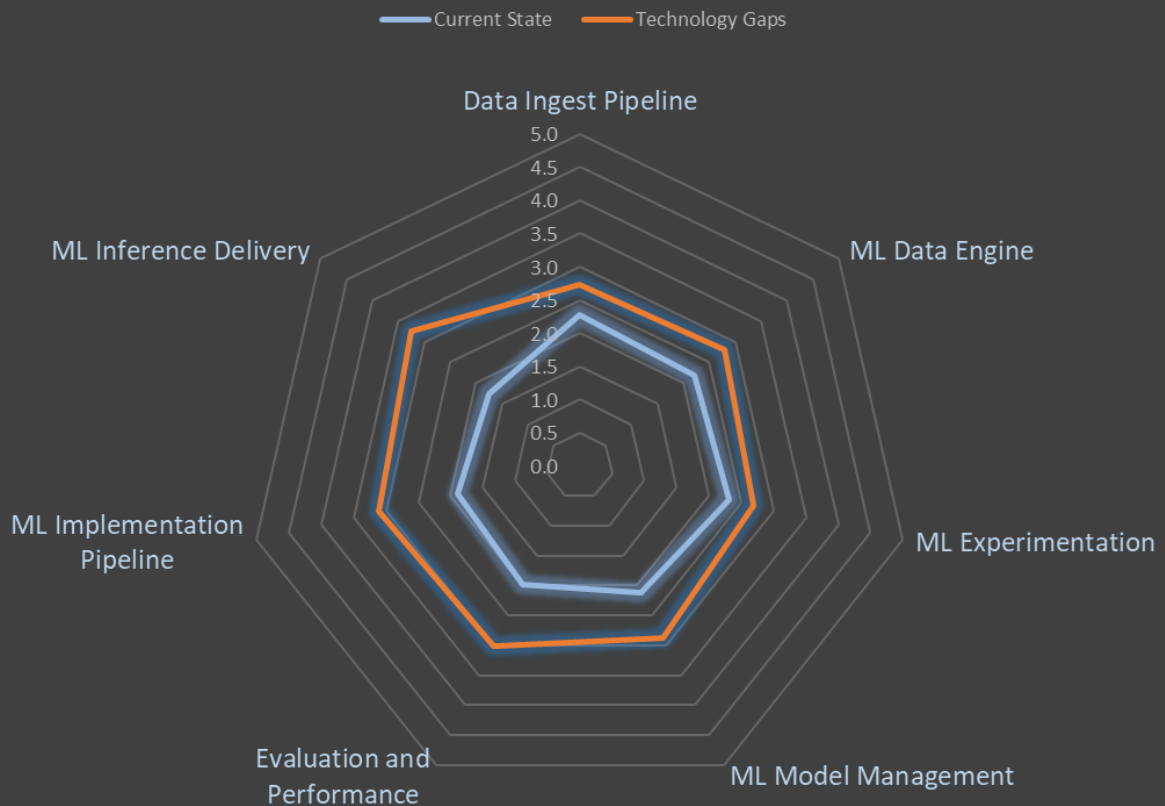
ASSESSMENT RESULTS

The results of each MLCC Assessment drive the identification of high, medium, and low priority tactical use cases to focus on during project delivery.

TECHNOLOGY:

Resolution Life has built strengths in several technology areas but requires additional support and capability development as pictured below:

Technology Assessment



RESOLUTION LIFE MLCC IMPLEMENTATION RECOMMENDATION

Resolution Life seeks to enhance their current platform to enhance their enterprise ML capability that embeds flexibility at the data access and model development levels, while ingraining standardization and repeatability at the ML engineering, model deployment, knowledge sharing, and organization levels. The keys to success require (i) developing repeatable processes to deliver models into production and realizing value for the business, (ii) implementing the overall governance and operating models for scaling ML capabilities beyond individual projects, (iii) embedding leading security practices to allow access to data and use of production model inference, and (iv) efficiently sharing knowledge across the global organization to maximize the return on every ML investment made. AWS assessed both Resolution Life's current state reality of ML capabilities delivered on a project by project basis, and the future state vision for an enterprise ML capability across the Technology pillar components of the MLCC delivery approach. The results of these assessments, in collaboration

with the Resolution Life team, informed the focus on the four MLCC building block use cases highlighted below.



SUCCESS METRICS AND BENEFITS

The following success metrics are the priority for Resolution Life. For the highest priority tactical use cases, Resolution Life aims to achieve these objectives:

1. **Model Development Performance:** Efficient development of models, adherence to production gating, frequency of reaching production, and realization of business value
2. **Cost Alignment/Optimization:** Costs of ML deployments are aligned with both business and Data Science objectives, across both latency and compute
3. **Enterprise Solution Costs:** The overall costs of implementing and maintaining the platform enterprise ML capability and centralized team.

DETAILED TACTICAL ML USE CASES

Resolution Life seeks to establish an enterprise ML capability and centralized team to create, deploy, and monitor quick-start ML environments, development standards, and deployment mechanisms that deliver value to business unit customers. These building block projects also support Resolution Life in reducing risk, streamlining training, and attaining value from the application of machine learning more rapidly. We recommend implementing the targeted

MLCC building block projects in parallel and with deploying production ML models to make the resulting process and best practice documentation, code, and guidance tangible for the Resolution Life team and broader community. AWS expert resources will assist partner Resolution Life Data Science resources to identify and deliver ML tasks and projects, develop ML artefacts, and conduct knowledge transfer.

MLCC TACTICAL USE CASE #10: DATA INGESTION PIPELINE

Resolution Life would like to enhance the existing ML data ingestion pipeline to 1) faster onboard new data sources 2) ensure higher quality data management and merging between different data sources and sensitive data, and 3) semi-automate feature generation. Resolution Life also needs to standardize methods to set-up new data pipelines.

Resolution Life is understood to already have an existing data lake and robust data ingestion pipeline in place. However, there is a need to enhance the data pipeline management to share source data across the organisation, at sufficient latency and in useful formats for ML modelling purposes, with streamlined data classification and ingest configuration capabilities. Also, while an extensive data lake exists and supports the development of new analytics, Resolution Life's Data Science teams currently manually query the data lake for their ML use cases on an ad-hoc basis. The recommendation is to build an ML specific data channel from the current data lake to make it easier to ingest data into a central ML platform that is able to scale.

OBJECTIVES

The overall goal of this initiative is to provide structures, processes, and policies for implementing an effective ML operating model, which include:

- Enhancing data definitions, dictionaries, and sensitive data management for ML use cases
- Standardizing and continually improve optimal data splits for production performance
- Discovering and addressing gaps for data source matching production vs. QA
- Develop the linkages to data sources in data lake and ML platform
- Develop data pre-processing templates to transform and quality check data for ingestion
- Create a data process that scales across use cases in a stable and performant manner
- Create standards for data pipeline deployment (e.g., Cloud formation templates, etc.)
- Develop a full lifecycle of data preparation for a limited set of data, which may include ingestion, parsing, integration, profiling, pruning, and feature generation
- Develop a feature store and cache management repository to store feature engineering data sets to support batch model predictions
- Develop a feature process to process real-time data to support live model predictions
- Create a data process that scales across use cases in a stable and performant manner
- Assess and solution the infrastructure required to support the engine at scale

MEASUREMENT METRICS

Measurement metrics could include the following:

- **Processing Performance**
- **Data Integration Flexibility:** Includes ease of deriving and testing new variables and sensitive data (PII) management
- **System Costs**
- **Number of concurrent AI/ML Research initiatives**

AWS SERVICE IMPLEMENTATION NOTES

The following elements from the AWS Well Architected Framework ML Lens will be relevant.

[Use a data lake house architecture](#) - Get the best insights from exponentially growing data using lake house data management architecture. It enables easy movement of data between data lake and purpose-built stores including: data warehouse, relational databases, non-relational databases, ML and big data processing, and log analytics. A data lake provides a single place to run analytics across mixed data structures collected from disparate sources. Purpose-built analytics services provide the speed required for specific use cases like real-time dashboards and log analytics.

- **Unify data governance and access** — Integrate a data lake, a data warehouse, and purpose-built stores. This will enable unified governance and easy data movement. With a [Lake House architecture](#) on AWS, you can store data in a data lake and use data services around it. Use [AWS Lake Formation](#) to build a scalable and secure data lake. Build a high-speed analytic layer with purpose-built services, such as [Amazon Redshift](#), [Amazon Kinesis](#), and [Amazon Athena](#). Integrate data across services and data stores with [AWS Glue](#). Apply governance policies to manage security, access control, and audit trails across all the data stores using [AWS IAM](#).

[Use a data catalog](#) - Process data across multiple data stores using data catalog technology. An advanced data catalog service can enable ETL process integration. This approach enables more reliability and efficiency.

- **Use AWS Glue Data Catalog** — The [AWS Glue Data Catalog](#) provides a way to track the data assets that have been loaded into your ML workload. Data catalogs not only track the assets but also describe how they are transformed as they are loaded into your data lake and data warehouse. AWS Glue is a fully managed ETL (extract, transform, and load) service. It enables simple and cost-effective approach to categorize your data, clean it, enrich it, and move it reliably between various data stores and data streams. AWS Glue consists of a central metadata repository known as the AWS Glue Data Catalog. It also has an ETL engine that automatically generates Python or Scala code. With a flexible scheduler, AWS Glue handles dependency resolution, job monitoring, and retries.

Use a data pipeline - Automates processing and movement and transformation of data between different compute and storage services. It enables data processing that is fault tolerant, repeatable, and highly available.

- **Use Amazon SageMaker Data Wrangler and Pipelines** — [SageMaker Data Wrangler](#) simplifies the preparation of machine learning data. It enables data selection, cleansing, exploration, and visualization, using a single visual interface. After you've created a workflow, export it to [SageMaker Pipelines](#) to automate model deployment and management. Data pipelines provide an automated way to move and transform data in your ML workload. Manually moving and transforming data can lead to errors and inconsistencies. Use [AWS Data Pipeline](#) to save time and decrease errors.

Protect sensitive data privacy of data used in training against unintended disclosure. Identify and classify the sensitive data. Handle the sensitive data using strategies including: removing, masking, tokenizing, and principal component analysis (PCA). Document best governance practices for future reuse and references.

- **Use automated mechanisms to classify data where possible** — Use systems such as [Amazon Macie](#) to classify and identify data locations and potentially sensitive data.
- **Use tagging** - Tag resources and models that are made from sensitive elements to quickly differentiate between resources requiring protection and those that do not.
- **Encrypt sensitive data** — Encrypt sensitive data using services such as [AWS KMS](#), the [AWS Encryption SDK](#), or client-side encryption.
- **Reduce data sensitivity** — Evaluate and identify data for anonymization or de-identification to reduce sensitivity.

Enable feature reusability - Reduce duplication and rerun of feature engineering code across teams and projects using a feature storage. The store should have online and offline storage, and data encryption capabilities. Online store with low-latency retrieval capabilities is ideal for real-time inference. Offline store should maintain a history of feature values and is suited for training and batch scoring.

- **Use Amazon SageMaker Feature Store** — [Amazon SageMaker Feature Store](#) is a fully managed, purpose-built repository to store, update, retrieve, and share ML features. Feature Store makes it easy for data scientists, machine learning engineers, and general practitioners to create, share, and manage features for ML development. Feature Store accelerates this process by reducing repetitive data processing and curation work required to convert raw data into features for training an ML algorithm.

You can use Feature Store in the following modes:

- Online — Features are read with low latency reads (milliseconds) and used for high throughput predictions. This mode requires a feature group to be stored in an online store.
- Offline — Large streams of data are fed to an offline store, which is used for training and batch inference. This mode requires a feature group to be stored in an offline store. The offline store uses your S3 bucket for storage and can also fetch data using Amazon Athena queries.
- Online and offline — This includes both online and offline modes.

MLCC TACTICAL USE CASE #11: ML DATA ENGINE

Resolution Life would like to establish well managed and governed processes around data management to increase the productivity of the data science team whilst reducing the risk of data related malpractice. Manage the data processing required around extraction, transformation and loading (ETL) in a consistent manner, sharing the same tooling across different ML use cases, in accordance with clearly defined guidance around (i) separation training, testing and validation datasets, (ii) governance of personally identifiable (PII) data, (iii) processes for the addition of new datasets to Research, QA and Production environments and (iv) control and monitoring of how data is promoted from Research to Production environments.

CHALLENGES

Resolution Life faces the following challenges in delivering in this area today:

- Lack of consistent guidance on experimental data usage and feature generation
- Lack of process to establish experimental data usage best-practices based on observed production performance
- Lack of uniform guidance regarding feature generation best practices and restrictions
- Lack of uniform processes ensuring key data downstream replication from Production to QA and Research environments in an automated, timely manner
- Lack of uniform processes and guidance on promoting Research data to QA or Production environments
- Lack of consistent processes to control, log, monitor, and audit data usage patterns in Research, QA and Production environments
- Lack of uniform access to reusable tools across teams/environments

OBJECTIVES

The overall goal of this initiative is to develop an effective ML Data Engine; specific objectives include:

- Define guidance and metrics for separating data into training, test, and validation datasets
- Establish guidance around the use of personally identifiable information (PII) or other sensitive data in ML environments/projects

- Establish processes to add new data sources to the Research environment is clearly defined, controlled, and monitored, and operates in a timely fashion
- Establish processes to add new data sources to the QA environment is clearly defined, controlled, and monitored, and operates in a timely fashion
- Establish processes to add new data sources to the Production environment is clearly defined, controlled, and monitored, and operates in a timely fashion
- Establish processes to escalate data from Research environment to Production environment in a controlled and monitored manner
- Provide transparency for every data source/dataset that is ingested into the data lake and make it easily discoverable without having to chase down the business owner;
- Storing, versioning and retrieving datasets for all phases of the ML lifecycle, including ideation/ experimentation, development and production, to support the documentation and reproducibility of ML models results;
- Enable efficient data sourcing by introducing the role of 'data producer', who are responsible for making data sharable and available;
- Enable efficient data discovery by introducing the role of 'data consumer' who are interested in discovering data, within defined access controls;
- Layout the foundations for scaling to a fully automated Data Marketplace in future phases;
- Design for data access being governed and secured via built-in controls so that workload teams can access the required data, at the appropriate granularity, for the appropriate times in an effective and efficient manner;
- Enable data lineage and transparency measures regarding data usage and propagation through different environments;
- Enable common data for development and production deployment so that there is alignment between the two.

MEASUREMENT METRICS

Measurement metrics could include the following:

- Velocity of Research: Decrease time of Research modeling by providing automated synchronization and environment setup
- Improved Production Performance: Empirically based data usage patterns seek to minimize performance slippage from Research to Production
- Risk Mitigation: Empirically based unit/stress testing patterns allow for identification and analysis of adverse scenarios and a-priori development of mitigation processes
- Time reduction to source data (and indirectly, time reduction in end-to-end model deployment process)
- Growth rate of data sources registered and utilised by ML teams
- Growth rate of newly populated features or pre-processing packages/pipelines
- Increased insights/control on edits to the original data set
- Quality of data ingested and re-usability

- Reduction of unnecessary dataset duplication

AWS SERVICE IMPLEMENTATION NOTES

The following elements from the AWS Well Architected Framework ML Lens will be relevant.

Profile data to improve quality Profile data to use data characteristics like distribution, descriptive statistics, data types, and data patterns. Review source data for content and quality. Filter out or correct any data not passing the reviews. This will contribute to quality improvement.

- **Use Amazon SageMaker Data Wrangler** — Import, prepare, transform, visualize, and analyze data with [SageMaker Data Wrangler](#). You can integrate Data Wrangler into your ML workflows to simplify and streamline data pre-processing and feature engineering with little to no coding. You can also add your own Python scripts and transformations to customize your data preparation workflow. Import data from [Amazon S3](#), [Amazon Redshift](#), or other data sources, and then query the data using [Amazon Athena](#). Use Data Wrangler to create sophisticated machine learning data preparation workflows with built-in and custom data transformations and analysis features. These features include feature target leakage and quick modeling.
- **Create an automatic data profile and a reporting system** — Use [AWS Glue Crawler](#) to crawl the data sources and create a data schema. Use [AWS Glue Data Catalog](#) to list all the tables and schemas. Use [Amazon Athena](#) for serverless SQL querying to constantly profile data and then use [Amazon QuickSight](#) dashboards for visualization of the data.

Document data processing - Document and version control any data processing-related findings, processes, and improvement to enable easier future referencing and reuse. Sample topics to document with respect to data processing include: sources of data, time of data collection, processes, scripts used for data collection, structure of data, types of transformation, and manipulations conducted on data.

Implementation plan

- **Associate notebook instances with Git repositories** — To analyze data, document its processing, and evaluate ML models on Amazon SageMaker, you can use [Amazon SageMaker Processing](#). SageMaker Processing can be used for feature engineering, data validation, model evaluation, and model interpretation. SageMaker notebook instances can be associated with Git repositories. This enables saving notebooks in a source control environment that persists after stopping or deleting the notebook instance. The notebooks will hold the data processing coding and its documentation. You can associate one default repository and up to three additional repositories with a notebook instance. The repositories can be hosted in [AWS CodeCommit](#), GitHub, or on any other Git server.

Use managed data labeling - Choose a managed labeling tool that provides automation and access to cost-effective labeling workforce. It should also provide flexibility to choose variable

number of labelers for a given input. The tool should have a user interface, and learn to label data by itself over time.

- **Use Amazon SageMaker Ground Truth** — To train a machine learning model, you need a large, highquality, labeled dataset. Amazon SageMaker Ground Truth helps you build high-quality training datasets for your ML models. With Ground Truth, you can use ML along with workers from Amazon Mechanical Turk, a vendor company that you choose, or an internal, private workforce to a labeled dataset. You can use the labeled dataset output from Ground Truth to train your own models. You can also use the output as a training dataset for an Amazon SageMaker model.

Ensure feature consistency across training and inference - Ensure consistent, scalable, and highly available features between training and inference using a feature storage. This results in reducing the training-serving skew by keeping feature consistency between training and inference.

- **Use Amazon SageMaker Feature Store** — Create, share, and manage features for ML development using [SageMaker Feature Store](#). The Feature Store is a centralized store for features and associated metadata so features can be easily discovered and reused. The online store is used for low latency, real-time inference use cases. The offline store is used for training and batch inference. The Feature Store reduces the repetitive data processing and curation work required to convert raw data into features for training an ML algorithm. Features generated will be used for both training and inference, reducing the training-serving skew. The Feature Store enables feature consistency, feature standardization, and the ability to integrate with [Amazon SageMaker Pipelines](#).

MLCC TACTICAL USE CASE #12: ML EXPERIMENTATION

Experimentation is an important aspect for extracting business value from data and machine learning, as it reduces uncertainty early on and with minimal effort. Before significant resources are mobilised on a potential use case, workload teams need to be able to estimate its success rate and business impact.

Resolution Life recognizes that the key to AI/ML success is repeatability. Maturity in the Experimentation segment has at its core concepts, repeatability of modelling runs and associated results. This is key to validate a range of critical issues in the AI/ML lifecycle – from model-to-model evaluation in Research, to unit/stress testing in QA, through to model performance in production. Repeatability is envisioned at the most granular level, as integrated (stored, versioned) code, model-dependent parameters/hyper-parameters, train, test & validation data (and any related transformations) used for a given generative cycle, the resulting model artefact (feature weights, etc.) and the associated results – all tightly coupled as a single “experiment.”

CHALLENGES

Resolution Life faces the following challenges in delivering in this area today:

- Lack of consistent approach with regard to experimentation atomicity
- Multiple tools allow a variety of approaches, lack of standardization
- Lack of clear guidance on best-practices for experimentation

OBJECTIVES

The overall goal of this initiative is to provide structures and processes to enable effective ML experimentation; specifics objectives include:

- Delivery of framework that leverages AI/ML tools to create integrated, atomic research artefacts across model code, data, data transformation, parameter/hyper-parameter iterations/experiments
- Delivery of a framework that leverages AI/ML tools that allow for repeatability in AI/ML experiments
- Delivery of a framework that leverages AI/ML tools that allow for sharing of model code, artefacts, and experimentation with other Data Scientists and other Data Science teams
- Delivery of a framework that embeds best practices around documentation/reference for AI/ML code & experiments
- Provide an easy-to-use sandbox environment for fast ideation and experimentation with pre-approved dataset.

MEASUREMENT METRICS

Measurement metrics could include the following:

- Velocity of Experimentation: The time required to research potential AI/ML solutions and alternative modeling approaches, as well perform due diligence
- Increased Collaboration: Reproducible atomic experiments and shared artefacts allow Data Scientists to collaborate seamlessly, produce transparent AI/ML solutions
- Risk Mitigation: Transparency in experimentation, repeatability in experimentation allows significantly better due diligence across Data Scientists

AWS SERVICE IMPLEMENTATION NOTES

The following elements from the AWS Well Architected Framework ML Lens will be relevant.

Identify if machine learning is the right solution. Evaluate if there are alternatives, such as a simple rule-based approach, that could do a better job than ML. Reasons for not selecting ML as a solution include: slower prediction time, lack of explainability, and insufficient data or features.

• **Use Amazon SageMaker Autopilot and SageMaker Clarify to validate that ML is the right solution**

- **Baseline the solution** by reviewing how the problem is solved today. If a rules-based solution is available, then use it as a baseline.

- **Build a machine learning model** using [SageMaker](#) or [SageMaker Autopilot](#) and compare the metrics of this solution against the baseline.
- **Use [SageMaker Clarify](#)** to explain the model that you have built using SageMaker or Autopilot.
- **Identify** if the ML model is performing better than your existing solution or a rules-based approach before investing on an ML-based solution.

[Understand and manage the available services and resources](#) - Identify the relevant services and configuration options for the workload. Understand how to achieve optimal performance. Select proper infrastructure that meet the workload requirements.

- **Learn about AWS AI services** — Determine whether [AWS managed AI services](#) are applicable to the business use case. Understand how managed AWS AI services can relieve the burden of training and maintaining an ML pipeline. Use [Amazon SageMaker](#) to develop in the cloud and understand the roles and responsibilities needed to maintain the ML workload. Consider combining managed AI services with custom ML models built on Amazon SageMaker. This will enable balancing the tradeoffs between ML workload management, and solutions specificity for the business use case.

[Select local training for small scale experiments](#) - Evaluate the requirements to train an ML model in the cloud versus on a local machine. Use local option when experimenting across different algorithms and configurations with small data sizes. For large data, launch a cloud-based training cluster with one or more compute instances. Right size the compute instances in the training cluster based on the workload.

Implementation plan

Use Amazon SageMaker — While experimenting with training a model with small datasets, use Amazon SageMaker notebook [local](#) mode. This will train your model on the notebook instance itself, instead of on a separate managed training cluster. You can iterate and test your work without having to wait for a new training or hosting cluster each time. This saves both time and cost associated with creating a managed training cluster.

During training, Amazon SageMaker retrieves the container you specify from [Amazon ECR](#) and prepares it to run on a training instance. For smaller data sets, Amazon SageMaker supports file mode for training, which downloads the training data from the [S3 bucket](#) to the [EBS volume](#) attached to the training instance. This allows the algorithm to read its training data from the local file system without integrating directly with Amazon S3. By using containers and copying objects from Amazon S3, Amazon SageMaker enables network isolation of your algorithms and models during training and hosting.

Experimentation can also occur outside of a notebook, for example on a local machine. From your local machine, you can use the [SageMaker SDK](#) to train and deploy models on AWS.

Review fairness and explainability - Consider fairness and explainability during each stage of the ML lifecycle. Compile a list of questions to review for each stage including:

- Problem framing — Is an algorithm an ethical solution to the problem?
- Data management — Is the training data representative of different groups? Are there biases in labels or features? Does the data need to be modified to mitigate bias?
- Training and evaluation — Do fairness constraints need to be included in the objective function? Does changing the number of models to train needed to mitigate bias? Has the model been evaluated using relevant fairness metrics?
- Deployment — Is the model deployed on a population for which it was not trained or evaluated?
- Monitoring — Are there unequal effects across users?

Use Amazon SageMaker Clarify — Understand model characteristics, debug predictions, and explain how ML models make predictions with [Amazon SageMaker Clarify](#). Amazon SageMaker Clarify uses a model-agnostic feature attribution approach that includes an efficient implementation of [SHAP](#) (Shapley Additive Explanations). SageMaker Clarify enables you to:

- Understand the compliance requirements for fairness and explainability.
- Measure biases that can occur during each stage of the ML lifecycle. These stages include data collection, model training, model tuning, and model monitoring.
- Generate model governance reports targeting risk and compliance teams and external regulators.
- Determine whether training data is biased in its classes or population segments, particularly protected groups.
- Develop a strategy for monitoring for bias in data when the model is in production.

Consider the trade-offs between model complexity and explainability, and select simpler models if explainability is required.

Use automated machine learning - Use automated data analyzer systems when building a model. These systems experiment with and select the best algorithm from the list of high-performing algorithms. They automatically test different solutions and parameter settings to achieve optimal models. The automated system speeds up the process, while eliminating manual experimentation and comparisons.

○ **Use Amazon SageMaker Autopilot** — [Amazon SageMaker Autopilot](#) automates key tasks of an automatic machine learning (AutoML) process. Autopilot explores your data, selects

the algorithms relevant to your problem type, and prepares the data to facilitate model training and tuning. Autopilot applies a cross-validation resampling procedure automatically to all candidate algorithms. It tests the ability of these algorithms to predict using data they have not been trained on. Autopilot simplifies your machine learning experience by automating the key tasks that constitute an AutoML process. It ranks all of the optimized models tested by their performance. Autopilot finds the best performing model that you can deploy at a fraction of the time normally required.

Define relevant evaluation metrics. To validate and monitor model performance, establish numerical metrics that directly relate to the KPIs. These KPIs are established in the business goal identification phase. Model accuracy might be an adequate metric for general model performance. Simple numerical metrics could be misleading in business applications depending on the use case. Evaluate whether the performance metrics accurately reflect the business' tolerance for the error. For instance, false positives might lead to excessive maintenance costs in predictive maintenance use cases. Numerical metrics such as precision and recall would help differentiate the business requirements and be closer aligned to business value. Consider developing custom metrics that tune the model directly for the business objectives. Examples of standard metrics for ML models include:

- **Classification**
 - Confusion matrix (precision, recall, accuracy, F1 score)
 - Receiver operating characteristic (ROC)-area under curve (AUC)
 - Logarithmic loss (log-loss)
- **Regression**
 - Root mean square error (RMSE)
 - Mean absolute percentage error (MAPE)
- **Optimize business-related metrics** — Identify performance metrics relevant to use-case and model type. Implement the metric as a loss function or use the loss function included in [Amazon SageMaker](#). Use [Amazon SageMaker Experiments](#) to evaluate the metrics with consideration to the business use case to maximize business value. Track model and concept drift in real time with [Amazon SageMaker Model Monitor](#) to estimate errors.
- Calculate the maximum probability of error that will be required for the ML model to produce results considering the tolerance set by the business.
- Select and train ML models on the available data to make prediction within the probability bounds. Organize tests on different models with Amazon SageMaker Experiments.

Ensure model validation with relevant data - Put processes in place to include real and representative data for testing and validation. Data that does not include all possible patterns and scenarios will result in failures once model is in production.

- **Use Amazon SageMaker Experiments** — Your models should be tested and validated using data that is representative of what they will encounter in production. This data can include both real-world data and engineered data. You should account for all scenarios in your training data so that you can avoid errors when your model is deployed to production. Use [Amazon SageMaker Experiments](#) to organize, track, compare, and evaluate your machine learning experiments.
- **Use Amazon SageMaker Model Monitor** — Consider implementing a plan to periodically test endpoints for deviations in model quality. Early detection of deviations can help you determine when to take corrective actions. [SageMaker Model Monitor](#) continually monitors the quality of Amazon SageMaker ML models in production. With Model Monitor, you can set alerts that notify you when there are deviations in the model quality.

Select optimal algorithms - Identify the basic machine learning paradigm that addresses your ML problem type. Basic machine learning paradigms include: supervised learning, unsupervised learning, and reinforcement learning. Identify the acceptable level of tradeoff between explainability and accuracy per business requirements. Run prototypes and experiments to explore high performing algorithms. Select the optimal cost-efficient algorithms that meet all the business requirements. Improved runtime performance of a tuned algorithm within business requirements, is one step towards optimizing the cost of ML.

- **Adopt optimal practices**

Start with simple algorithms, such as regression, and work towards more complex algorithms, such as deep learning, to compare the accuracy of the models. Optimize hyperparameters to determine which algorithm yields the best metrics for the business use case.

When selecting the optimal algorithm, run trade-off analysis between data constraints, computational performance, and maintenance efforts. For example, deep learning networks might produce more accurate results, but require more data than tree-based methods. Deep learning methods are also more difficult to maintain.

- **Use AWS services**

- Use Amazon SageMaker with a suite of built-in algorithms to train and deploy machine learning models. AWS provides optimized versions of frameworks, such as TensorFlow, Chainer, Keras, and Theano. These frameworks include optimizations for high-performance training across Amazon EC2 instance families.
- Use Amazon SageMaker Experiments to keep track of the models during testing.
- Use Amazon SageMaker Autopilot to select algorithms automatically.

[Tradeoff analysis on custom versus pre-trained models](#) - Optimize the cost through tradeoff analysis based on custom versus pre-trained models. This tradeoff analysis should keep the security and performance efficiency in perspective and within the acceptable thresholds.

- Use Amazon SageMaker built-in algorithms and AWS Marketplace — Amazon SageMaker provides a suite of built-in algorithms to help data scientists and machine learning practitioners get started on training and deploying machine learning models. Pre-trained ML models are ready-to-use models that can be quickly deployed on Amazon SageMaker. By pre-training the ML models for you, solutions in the AWS Marketplace take care of the heavy lifting, helping you deliver AI- and ML-powered features faster and at a lower cost. Evaluate the cost of your data scientists' time and other resource requirements to develop your own custom model vs. bringing a pre-trained model and deploying it on SageMaker for inferencing. The advantage of a custom model is the flexibility to fine-tune it to match the needs of your business use case. A pre-trained model can be difficult to modify and you might have to use it as is.

[Use hyperparameter optimization technologies](#) - Use automatic hyperparameter tuning to run many training jobs and find the best version of your model. Use the algorithm and ranges of hyperparameters that you specify. Use appropriate hyperparameter ranges, as well as metrics that are realistic and meet the business requirements.

Implementation plan

Use SageMaker automatic model tuning — [SageMaker automatic model tuning](#), also known as hyperparameter tuning, finds the optimal model by running many training jobs on your dataset. It uses the algorithm and ranges of hyperparameters that you specify. It then chooses the hyperparameter values that result in a model that performs the best, as measured by a metric that you choose. To create a new [hyperparameter optimization](#) (HPO) tuning job for one or more algorithms, you need to define the settings for the tuning job. Create training job definitions for each algorithm being tuned, and configure the resources for the tuning job.

[Enable debugging and logging](#) - Ensure that there are sufficient logs and metrics recorded to capture the runtime and resource consumption. The collected logs and metrics can be analyzed to identify the areas for improvement. Monitor compute and data storage consumption.

Instrument the machine learning code, and use debugging tools to capture metrics at runtime.

- Use Amazon SageMaker Debugger — [Amazon SageMaker Debugger](#) captures the state of a training job at periodic intervals. It provides visibility into the ML training process by monitoring, recording, and analyzing data with the ability to perform interactive exploration of data captured during training. The debugger has an alerting capability for errors detected during training. For example, it can automatically detect and alert you to commonly occurring errors, such as gradient values getting too large or too small.

- Use Amazon CloudWatch — Logs generated during training by Amazon SageMaker are logged to [Amazon CloudWatch Logs](#). Use an [AWS KMS key](#) to encrypt log data ingested by Amazon CloudWatch Logs.

Validate ML software privacy and license terms. ML libraries and packages handle data processing, model development, training, and hosting. Establish a process to review the privacy and license agreements for all software and ML libraries needed throughout the ML lifecycle. Ensure these agreements comply with your organization's legal, privacy, and security terms and conditions. These terms should not add any limitations on your organization's business plans.

- **Implement a package mirror for consuming approved packages** — Evaluate the license terms to determine which ML packages are appropriate for your business across the phases of the ML lifecycle. Examples of ML Python packages include: Pandas, PyTorch, Keras, NumPy, Scikit-learn. Once you've determined the set and criteria, build a validation mechanism and automate it where possible. A sample automated mechanism can include a script that runs the download, installation, and package version and dependency checks. [Download packages from the internet, only from approved and private repos](#). Validate what is in the packages downloaded. This will enable importing safely and confirming the validity of packages. [Amazon SageMaker notebook instances](#) come with multiple environments already installed. These environments contain Jupyter kernels and Python packages. You can also install your own environments that contain your choice of packages and kernels. SageMaker enables [modifying package channel paths to a private repository](#). Where appropriate, use an internal repository as a proxy for public repositories to minimize the network and time overhead.
- **Bootstrap instances with lifecycle management policies** — Create a lifecycle configuration with a reference to your package repository, and a script to install required packages.
- **Evaluate package integrations that require external lookup services** — Based on your data privacy requirements, opt out of data collection when necessary. Minimize data exposure through trusted relationships. Evaluate the privacy policies as well as the license terms for ML packages that might collect data.
- **Use prebuilt containers** — Start with pre-packaged and verified containers to quickly provide support for commonly used dependencies. For example, [AWS Deep Learning Containers](#) contain several deep learning framework libraries and tools including TensorFlow, PyTorch, and Apache MXNet.

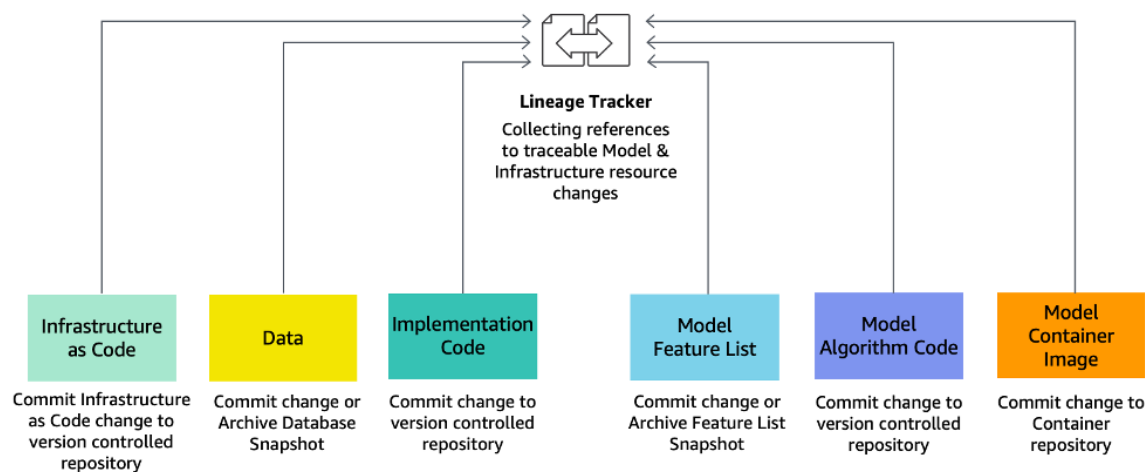
ML best practices include guidance on how to manage models and their dependencies throughout their lifecycle. This is particularly crucial in ML, as model artefacts are subject to constant change of incoming real-life data while needing to maintain the initially approved performance criteria. Furthermore, regulatory and legal controls, such as GDPR or Basel Accords, require ongoing transparency into model dependencies, usage, and quantifiable performance metrics. In addition to these requirements, Resolution Life has their own internal quality assurance process for data models, which has proven to be useful in preventing low quality models entering production. ML model quality criteria span a variety of classes, starting with business metrics (e.g. customer engagement), model performance metrics (e.g. accuracy, f1 score etc.), extending to technical measures like latency and throughput, or data related measures such as model drift.

Resolution Life should enhance the ML Model Management Framework to assist with monitoring, measuring, and managing models pre- and post-deployment. To achieve this, Resolution Life will need to design and implement a Continuous Integration (CI)/Continuous Delivery (CD) pipeline that can enable industrialised deployment and management of ML models (even in QA) and this should be integrated into the Customer's existing CI/CD and production readiness processes. The CI/CD pipeline should consist of core functions to confirm code quality, compliances, test the ML code, enable data lineage and code version reviews, retrain existing ML models in production, determine strategic direction, and enable model roll-back and auditing.

The ML lineage tracker collects references to traceable data, model and infrastructure resource changes. It consists of the following components:

- System architecture (infrastructure as code to address environment drift)
- Data (metadata, values, and features)
- Model (algorithm, features, parameters, and hyperparameters)
- Code (implementation, modeling, and pipeline)

The lineage tracker collects changed references through alternative iterations of ML lifecycle phases. The following figure shows the machine learning components and their information that the lineage tracker collects across different releases. The collected information enables going back to a specific point-in-time release and recreate it.



Lineage tracker components include:

- **Infrastructure as code (IaC)** — Modeling, provisioning, and managing cloud computing resources (compute, storage, network, and application services) can be automated using infrastructure as code. Cloud computing takes advantage of virtualization to enable the on-demand provisioning of resources. IaC eliminates configuration drift through automation, while increasing the speed and agility of infrastructure deployments. IaC code changes are committed to version-controlled repository.
- **Data** — Store data schemes and metadata in version control systems. Store the data in a storage media like a data lake. The location or link to the data can be in a configuration file and stored in code version control media.
- **Implementation code** — Changes to any implementation code at any point-in-time can be stored in version control media.
- **Model feature list** — Feature store technology referenced in the scenario architecture diagrams stores features as well as their versions for any point-in-time changes.
- **Model algorithm code** — Changes to any model algorithm code at any point-in-time can be stored in version control media.
- **Model container image** — Versions of model container images for any point-in-time changes can be stored in container repositories managed by container registry.

While the current practice is to make sure code repositories exist, the core functions of model management in best-in-class companies go beyond that to deliver elements like:

- **Model Measurement & Monitoring:**
 - Clearly defined guidance and metrics for determining model drift and data drift
 - Target metric (accuracy, precision, recall, f1, etc.) is used consistently over time
 - Processes and metrics exist defining impact of slippage of all production models' associated fitness metrics

- Model performance in production environment is monitored/logged, Impact of adverse performance is monitored and logged, and matches expectations from research developed in research/QA
- **Model Assessment & Retraining:**
 - Subsequent retraining captures/logs feature importance of each retrained model
 - Periodic audits to evaluate model performance and feature importance against historical/research performance and feature importance
 - Processes and metrics governing re-training of production models
 - Processes and metrics governing termination/off-lining of production models
- **Model Governance:**
 - Guidance, processes, and logging for any changes made to production models
 - Process to add new data sources to the Production environment is clearly defined, controlled, and monitored, and operates in a timely fashion
- **Model Repository and Feature Stores**
 - Versioning of the model artifacts including – model file (binaries), training data objects, config files etc.
 - Versioning of data (individual records, engineered features, metadata), across iterations is essential for consistent model outcome reproducibility.
 - Define model catalogue criteria, especially with respect to regulatory requirements;
 - The repository should also be easily accessible and monitored in order for the libraries to be re-used more frequently.
- **ML Workload Characteristics:**
 - Model deployment sample characteristics include:
 - Model size (model.tar.gz) in bytes
 - Number of models deployed per endpoint
 - Instance size (AWS example: r5dn.4x.large)
 - Model/endpoint update schedule (hourly, daily, weekly, monthly, per-event)
 - Model deployment location (on-premises, Amazon EC2, container, serverless, edge)
 - Architectural deployment sample characteristics for the internal underlying algorithm or neural architecture includes:
 - Inference pipeline architecture (single endpoint, chained endpoints)
 - Neural architecture (single framework (Scikit-learn), multi framework (PyTorch + Scikit-learn + TensorFlow))
 - Containers (SageMaker prebuilt container, bring your own container)
 - Location of the containers and models (on-premises, cloud, hybrid)
 - Traffic pattern deployment sample characteristics include:
 - Traffic pattern (steady, spiky)
 - Input size (number of bytes)
 - Latency (low, medium, high, batch)

- Concurrency (single thread, multi-thread)
- Cold start tolerance characteristics — Determine and document the tolerance of the various aspects of cold start in milliseconds.
- Network deployment characteristics — Check for the applicability of network deployment characteristics including AWS KMS encryption, multi-variant endpoints, network isolation, and third-party Docker repositories.
- Cost considerations — Discuss and document the cost considerations for elements, such as Amazon Spot Instances.
- Determine provisioning matrix — Critical ML workloads might be vying for resources from cloud providers. For staging and production environments, include a matrix of the expected capacity requirements. This matrix consists of the number of instance types per AWS Region across training, batch inference, real-time inference, and notebooks.

OBJECTIVES

The overall goal of this initiative is to provide a process and system which would help:

- Deploy a model management system which showcases a robust model management process and repository
 - E.g., captures models, modelling outputs, scores, etc.
- Control access (and enforce compliance) to and versioning of ML models, data and codes
- Creates standards for baseline model selection and deployment
 - E.g., Cloud formation templates, Service catalogues, etc.
- Create a logging info and notification process (e.g., for when models pass through key stages in the Pipeline)
- Providing an appropriate basis for readily available input into regulatory, auditing and compliance processes;
- ML algorithm pipeline / library development
- Enable an automated retraining (if the use case allows)
- Enable efficient storage and retrieval of data snapshots across different iterations
- Develop a reusable framework and template standardisation of ML workloads for monitoring, logging, alerting, etc.

MEASUREMENT METRICS

Measurement metrics could include the following:

- **Processing Performance:** CI/CD Pipeline and overall Model Management Framework performance in processing ML models into production

- **Flexibility in Deployment (Real-Time & Batch):** Ability to deploy models in both real-time and batch inference functions
- **Velocity of Deployment and Retraining:** The time required to deploy a ML model, deliver inference to target users of the model, and maintain and retrain the model

AWS SERVICE IMPLEMENTATION NOTES

The following elements from the AWS Well Architected Framework ML Lens will be relevant.

[Establish a lineage tracker system](#) - Maintain a system that tracks changes for each release. These changes include documentation, environment, model, data, code, and infrastructure. Having this system allows you to go back and quickly reproduce a problem on a prior release, allowing rollbacks and reproducibility.

- **Identify artifacts needed for tracking** — Tracking all the artifacts used for a production model is an essential requirement for reproducing the model to meet regulatory and control requirements. [Data and artifacts lineage tracking](#) includes the list of artifacts needed for tracking.
- **Use SageMaker ML Lineage Tracking** — [SageMaker ML Lineage Tracking](#) creates and stores information about the steps of an ML workflow from data preparation to model deployment. With the tracking information, you can reproduce the workflow steps, track model and dataset lineage, and establish model governance and audit standards.
- **Use SageMaker Studio** — Use [SageMaker Studio](#) to track the lineage of an Amazon SageMaker ML pipeline.
- **Use SageMaker Model Registry** — Use [SageMaker Model Registry](#) to catalog models for production, manage model versions, and associate metadata with a model. Model Registry enables lineage tracking.
- **Use SageMaker Pipelines for model building** — With [SageMaker Pipelines](#) you can track the history of your data within the pipeline execution. [SageMaker ML Lineage Tracking](#) lets you analyze input data, its source, and the outputs generated.

[Detect transfer learning risk](#) - Monitor and ensure the inherited prediction weights from a transferred model lead to the correct results. This helps minimize the risk of weak learning and incorrect outputs using pre-trained models.

- **Leverage Amazon SageMaker Debugger** — Transfer learning is a machine learning technique where a model pre-trained on one task is fine-tuned on a new task. When using the transfer learning approach, use [Amazon SageMaker Debugger](#) to detect hidden problems that might have serious consequences. Examine model predictions to see what mistakes were made. Validate the robustness of your model, and consider how much of this robustness is from the inherited capabilities. Validate input and preprocesses to the model for realistic expectations.

MLCC TACTICAL USE CASE #14: EVALUATION AND PERFORMANCE

Resolution Life would like to design and implement a mechanism to verify and validate key metrics essential for high quality outcome from machine learning pipeline. Broadly the evaluation and performance can be done across these dimensions - machine learning model, the data and also with regard to the platform itself. Timely and precise identification of deviations of desired metrics across these dimensions can help to take preventive actions and maintain an overall healthy state of the pipeline with consistent results.

It is understood that Resolution Life faces the following challenges in this area today:

- **Model evaluation:** Model performance in production environment is monitored/logged. Impact of adverse performance is monitored and logged, and matches expectations from research developed in research/QA. Identification of drift across model performance evaluation metric. Constant check and evaluation of model Interpretability, which can act as a barrier for deriving actionable insights. Identification of model bias over a period of time in production. Optimal frequency for model retraining, which may need analytic over analytic to get best results.
- **Accuracy versus complexity trade-off:** The simpler a machine learning model is, the more explainable are its predictions. Deep learning predictions can potentially outperform linear regression or a decision tree algorithm, but at the cost of added complexity in interpretability and explainability.
- **Bias versus fairness trade-off:** Define a process for managing risks of bias and fairness in model performance. Business value most often aligns with models that have considered historical or sampling biases in the training data. Further consideration should be given to the disparate impact of inaccurate model predictions. For example, underrepresented groups are often more impacted by historical biases, which might perpetuate unfair practices.
- **Bias versus variance trade-off (supervised ML):** The goal is to achieve a trained model with the lowest bias versus variance tradeoff for a given dataset.
- **Data evaluation:** Data Identification of drift in the input data (in production). Validating data quality in terms of sufficiency and consistency, required for model development phase. And benchmarking the same to make the desired Golden dataset available for the specific task on time. Also, ensuring the required level data encryption and regulations are following consistently.
- **Platform performance:** A way to measure and notify any deviation from desired resource utilization% (CPU/Memory) for all the jobs in real time. Cost incurred in a periodic basis and evaluation scope for optimizing the same, latency in dynamic resource provisioning, lead time incurred to make the desired data available for inference.
- **Model Governance:** Guidance, processes, and logging exist for any changes made to production models.

OBJECTIVES

The overall goal of this initiative is to provide structures, processes, and policies for implementing an ML Pipeline evaluation mechanism that would help:

- Create a process to evaluate key components of the overall ML platform
- Enable ease of tracking the different metrics across models, data and platform in real time with appropriate notifications/alerts.
- Deploying a model management system which showcases a robust model management process and repository
 - E.g., captures models, modelling outputs, scores, etc.
- Creates standards for deployment
 - E.g., Cloud formation templates, Service catalogues, etc.
- Creates a logging info and notification process (e.g., for when models pass through key stages in the Pipeline)
- Enables an automated retraining (if the use case allows)
- Develops a reusable framework and template standardisation of ML workloads for monitoring, logging, alerting, etc.
- Create role-based access to the ML platform performance evaluation interface, for
 - Data Science
 - Machine Learning Engineers
 - Data Engineers
 - Data Owners
 - DevOps Developers

MEASUREMENT METRICS

ML platform performance evaluation metrics could include the following:

- **Velocity of Monitoring and Maintainability:** The time required to monitor and maintain a ML model
- **Model evaluation:** on time and accurate notification of key metrics finalized
- **Data evaluation:** Proactive notification of data quality metrics at appropriate phase in the pipeline.
- **Platform performance:** on time and accurate notification of key metrics finalized

AWS SERVICE IMPLEMENTATION NOTES

The following elements from the AWS Well Architected Framework ML Lens will be relevant.

Establish feedback loops across ML lifecycle phases - Establish a feedback mechanism to share and communicate successful development experiments, analysis of failures, and operational activities. This facilitates continuous improvement on future iterations of the ML workload. Feedback loops allow experimentation with alternative approaches and algorithms until an

optimal outcome is achieved. Document your findings to identify key learnings and improve processes over time.

- **Establish SageMaker Model Monitoring** — [Amazon SageMaker Model Monitor](#) continually monitors machine learning models for concept drift. Drifts include changes in data distribution and characteristics over time. SageMaker Model Monitor alerts you if there are any deviations so that you can take remedial action.
- **Use Amazon CloudWatch** — Configure [Amazon CloudWatch](#) to receive notifications if drift in model quality is observed. Monitoring jobs can be scheduled to run at a regular cadence (for example, hourly or daily) and push reports as well as metrics to [Amazon CloudWatch](#) and [Amazon S3](#).
- **Initiate pipelines** — Create a [CloudWatch Events](#) rule that alerts on an event emitted by SageMaker Model Monitoring system. The event rule can detect the drifts or anomalies, and start a retraining pipeline.
- **Use Amazon Augmented AI (A2I)** — Check accuracy by having human reviews using tools such as [Amazon A2I](#).

[Establish a model performance evaluation pipeline](#) - Capture key metrics related to model performance using an end-to-end performance pipeline to evaluate the success of a model. Choose specific metrics based on the use case and the business KPIs. Sample key metrics include training or validation errors, and prediction accuracy. Specific model performance metrics include RMSE, accuracy, precision, recall, F1 score, and AUROC. Establish a fully automated performance testing pipeline system to initiate evaluation every time there is an updated model or data.

- **Create an end-to-end workflow with Amazon SageMaker Pipelines** — Start with a workflow template to establish an initial infrastructure for model training and deployment. [SageMaker Pipelines](#) helps you automate different steps of the ML workflow. These steps include data loading, data transformation, training, tuning, and deployment. With SageMaker Pipelines, you can share and reuse workflows to recreate or optimize models, helping you scale ML throughout your organization. Within SageMaker Pipelines, the [SageMaker Model Registry](#) tracks the model versions and respective artifacts. These artifacts include the metadata and lineage data collected throughout the model development lifecycle. SageMaker Model Registry can also enable automating model deployment with CI/CD.
- **Use AWS MLOps Framework** — [AWS MLOps Framework](#) provides a standard interface for managing ML pipelines for [Amazon Machine Learning services](#) and third-party services. The solution's template allows you to upload your trained models (also referred to as *bring your own model*). It configures the orchestration of the pipeline, and monitors the pipeline's operations. This solution increases agility and efficiency by allowing repeating of successful processes at scale. One of the key components of MLOps pipeline in SageMaker is Model

Registry. [SageMaker Model Registry](#) tracks the model versions and respective artifacts, including the lineage and metadata.

Evaluate model explainability - Evaluate model performance as constrained by the explainability requirements of the business. Compliance requirements, business objectives, or both might require that the inferences from a model be directly explainable. Evaluate the explainability needs, and the trade-off between explainability and model complexity. Then select the model type or evaluation metrics. This approach provides transparency into the reasons that a particular inference was attained given the input data.

- **Use Amazon SageMaker Clarify to explain model results** — [Amazon SageMaker Clarify](#) helps improve your ML models by detecting potential bias and helping explain the predictions that models make. It helps you identify various types of bias in data that can emerge during model training or in production. SageMaker Clarify helps explain how these models make predictions using a feature attribution approach. It also monitors inferences that the models make in production for bias or feature attribution drift. The fairness and explainability functions provided by SageMaker Clarify helps you build less biased and more understandable machine learning models. It also provides tools to help you generate model governance reports that you can use to inform risk and compliance teams, and external regulators.

Evaluate data drift - Understand the effects of data drift on model performance. In cases where the data has drifted, the model could generate inaccurate predictions. Consider a strategy that monitors and adapts to data drift through re-training.

- **Use Amazon SageMaker Model Monitor, and SageMaker Clarify** — [Amazon SageMaker Model Monitor](#) helps you maintain high-quality ML models by detecting model and concept drift in real time, and sending you alerts so you can take immediate action. Model and concept drift are detected by monitoring the quality of the model. Independent variables (also known as features) are the inputs to an ML model, and dependent variables are the outputs of the model. Additionally, SageMaker Model Monitor is integrated with [Amazon SageMaker Clarify](#) to help you identify potential bias in your ML models.

Include human-in-the-loop monitoring - Use human-in-the-loop monitoring to monitor model performance efficiently. When automating decision processes, the human labeling of model results is a reliable quality test for model inferences. Compare human labels with model inferences to estimate model performance degradation. Perform mitigation as model re-training.

- **Use Amazon Augmented AI to get human review** — Learn how to design a quality assurance system for model inferences. Establish a team of subject matter experts to audit model inference in production. Use [Amazon Augmented AI](#) (Amazon A2I) to get human review of low-confidence predictions or random prediction samples. Amazon A2I uses resources in [IAM](#), [SageMaker](#), and [Amazon S3](#) to create and run your human review workflows.

Monitor, detect, and handle model performance degradation - Model Performance could degrade over time for reasons including: data quality, model quality, model bias, and model explainability. Continuously monitor the quality of the ML model in real time. Identify the right time and frequency to retrain and update the model. Configure alerts to notify and initiate actions if any drift in model performance is observed.

- **Monitor model performance** — [Amazon SageMaker Model Monitor](#) continually monitors the quality of Amazon SageMaker machine learning models in production. Establish a baseline during training before model is in production. Collect data while in production and compare changes in model inferences. Observations of drifts in the data statistics will indicate that the model may need to be retrained. The timing of drifts will establish a schedule for retraining. Use [SageMaker Clarify](#) to identify model bias. Configure alerting systems with [Amazon CloudWatch](#) to send notifications for unexpected bias or changes in data quality.
- **Perform automatic scaling** — [Amazon SageMaker](#) automatically monitors core system metrics. It includes capabilities to establish automatic scaling for your hosted model to dynamically adjust underlying compute supporting an endpoint based on demand. This capability ensures that your endpoint can dynamically support demand while reducing operational overhead.
- **Monitor endpoint metrics** — Amazon SageMaker also outputs endpoint metrics for monitoring the usage and health of the endpoint. Amazon SageMaker Model Monitor provides the capability to monitor your ML models in production and provides alerts when data quality issues appear. Create a mechanism to aggregate and analyze model prediction endpoint metrics using services, such as [Amazon OpenSearch Service](#) (OpenSearch Service). OpenSearch Service supports [Kibana](#) for dashboards and visualization. The traceability of hosting metrics back to versioned inputs allows for analysis of changes that could be impacting current operational performance.

Enable monitoring health of model endpoint - Establish model endpoint monitoring. Identify and react to any potential issues or opportunities for improvement. Monitor metrics that measure the operational health of the underlying compute resources hosting the endpoint and the health of endpoint responses. Ensure traceability of hosting metrics back to versioned inputs for analysis.

- **Use Amazon SageMaker Model Monitor** — Continually monitor the quality of Amazon SageMaker ML models in production and compare with the results from training using [SageMaker Model Monitor](#).
- **Use Amazon CloudWatch** — Amazon SageMaker Model Monitor automatically sends metrics to [Amazon CloudWatch](#) so that you can gather and analyze usage statistics for your ML models.
- **Use Amazon SageMaker Clarify** — Identify various types of bias that can emerge during model training or when the model is in production. This helps improve your ML models by detecting potential bias and helping explain the predictions that the models make. [SageMaker Clarify](#) helps explain how these models make predictions using a feature

attribution approach. It also monitors inferences that the models make in production for bias or feature attribution drift. SageMaker Clarify provides tools to help you generate model governance reports that you can use to inform risk and compliance teams, and external regulators.

Synchronize architecture and configuration, and check for skew across environments - Ensure all systems and configurations are identical across development and deployment phases. Otherwise, the same algorithm can result in different inference results depending on differences in system architectures. Ensure that the model gets the same range of accuracy in development, staging, and production environments. Perform this check as part of the normal promotion process.

- **Use AWS CloudFormation** — [AWS CloudFormation](#) gives you an easy way to model a collection of related AWS and third-party resources. CloudFormation provisions these resources quickly and consistently, and manages them throughout their lifecycles, by treating infrastructure as code. You can use a CloudFormation template to create, update, and delete an entire stack as a single unit, as often as you need to, instead of managing resources individually. You also can manage and provision stacks across multiple AWS accounts and AWS Regions. This will synchronize your architecture and configuration across environments.
- **Use Amazon SageMaker Model Monitor** — Continually monitor the quality of Amazon SageMaker machine learning models in production and compare with the results from training using [SageMaker Model Monitor](#). Set alerts that notify you when there are deviations in the model quality. Early and proactive detection of these deviations enables you to take corrective actions. These actions can include retraining models, auditing upstream systems, and fixing quality issues without having to monitor models manually or build additional tools. Model Monitor provides monitoring capabilities that do not require coding. You also have the flexibility to monitor models by adding code to provide custom analysis.

Establish feature statistics - Establish key statistics to measure changes in the data that affect model outcomes. The effect of data changes on model inferences depends on the sensitivity of the model to data features. Analyze the feature importance and sensitivity of the model to select the features to monitor. Monitor the statistics of features that most strongly affect inferences. Place acceptability limits on the range of data to alert when important features drift outside the statistical range of the training data. Significant drifts in important features would suggest model re-training.

- **Analyze and evaluate data** — Use [Amazon SageMaker Data Wrangler](#) to analyze the distribution of the selected features. After training the model, map out the regions in feature space where the predictions change abruptly and where the predictions are invariant. Establish a baseline for monitoring the data with [Amazon SageMaker Model Monitor](#). Perform a sensitivity analysis of changes in the feature values near the decision boundaries of the model. Analyze the feature importance to understand how new data will affect the model's predictions. [Amazon SageMaker Experiments](#) will help to organize model testing. Use

[Amazon SageMaker Clarify](#) to check for data biases and imbalances. Monitor the statistics of data used in production inferences. Consider retraining the model if the features are outside the original distribution of the training data.

MLCC TACTICAL USE CASE #15: ML IMPLEMENTATION PIPELINE

An analysis of AI/ML best practices across industries recognizes reusable ML Pipelines as an additional competency accelerating Data Science productivity. Reusable pipelines could provide efficiencies across a wide variety of AI/ML projects and Data Scientists to Resolution Life. Reusability also drives the development of templated resources to standardize deployment, monitoring and auditing of Resolution Life's AI/ML domain resources. Pipelines allow AI/ML to move from ad-hoc, operator/Data Scientist-heavy processes, to streamlined, automated, and scalable AI/ML for Resolution Life.

The core functions to deliver this use case include:

- Pipeline Features
 - ML Pipeline containing tools for environment setup
 - ML Pipeline containing tools for data access
 - ML Pipeline containing tools for data extraction, transformation, and loading (ETL)
 - ML Pipeline containing available algorithms
 - ML Pipeline containing methods for experiment management/artefact generation
 - ML Pipeline containing tools for generating integrated, scalable inference
 - Model artefacts (ETL, algorithm, and parameters) that are promoted directly across environments (Research to QA, QA to Production)
 - ML Pipeline tools that are templated, and reusable across/within groups

The effort will entail creating standardised tools and procedures that enable rapid building and testing of ML models as well as establishing the infrastructure, framework, and tooling to enable data scientists with self-service access to model development capabilities and source data pipelines. In addition, Resolution Life would like to design and implement ML platform foundations to enable the industrialised deployment and management of ML models, including a review and assessment of existing model deployment strategies. This should consist of core functions to confirm code quality, test the ML code, enable data lineage and code version reviews, retrain existing ML models in production, determine strategic direction and enable model roll-back and auditing.

OBJECTIVES

The overall goal of this initiative is to provide a process and system that would help in:

- Deploying a system which showcases the full lifecycle of training and deploying models for a discrete use case
- Ability to scale across multiple use cases in a stable and performant manner
- Delivery of framework that leverages AI/ML tools to create integrated, atomic research artefacts across model code, data, data transformation, parameter/hyper-parameter iterations/experiments
- Delivery of a framework that leverages AI/ML tools that allow for repeatability in AI/ML experiments
- Delivery of a framework that leverages AI/ML tools that allow for sharing of model code, artefacts, and experimentation with other Data Scientists and other Data Science teams; Create a sharing and managing mechanism for standardized notebooks for effective experimentation during data and model discovery
- Delivery of a framework that embeds best practices around documentation/reference for AI/ML code & experiments

MEASUREMENT METRICS

Measurement metrics could include the following:

- **Velocity of Experimentation:** The time required to research potential AI/ML solutions and alternative modeling approaches, as well perform due diligence; Number of concurrent AI/ML initiatives across QA and production; Speed of Deployment
- **Increased Collaboration:** Reproducible atomic experiments and shared artefacts allow Data Scientists to collaborate seamlessly, produce transparent AI/ML solutions
- **Risk Mitigation:** Transparency in experimentation, repeatability in experimentation allows significantly better due diligence across Data Scientists

AWS SERVICE IMPLEMENTATION NOTES

The following elements from the AWS Well Architected Framework ML Lens will be relevant.

Automate managing changes - using version control technology. This will enable reproducibility to re-create the exact version of a model in the event of a failure.

- **Use AWS MLOps Framework** — [AWS MLOps Framework](#) provides a standard interface for managing ML pipelines for [Amazon Machine Learning services](#) and third-party services. The solution's template allows you to upload your trained models (also referred to as *bring your own model*). It configures the orchestration of the pipeline, and monitors the pipeline's operations. This solution increases agility and efficiency by allowing repeating of successful processes at scale. One of the key components of MLOps pipeline in SageMaker is Model Registry. [SageMaker Model Registry](#) tracks the model versions and respective artifacts, including the lineage and metadata.

Automate operations through IaC and CaC - Automate ML workload operations using infrastructure as code (IaC) and configuration as code (CaC). This ensures consistency across

staging, and production deployment environments. Enable traceability and version control across infrastructures and configurations.

Implementation plan

- **Use AWS CloudFormation** — [AWS CloudFormation](#) enables you to create and provision AWS deployments predictably and repeatedly. AWS CloudFormation enables you to use a template file to create and delete a collection of resources together as a single unit (a stack). You can manage and provision stacks across multiple AWS accounts and AWS Regions.
- **Use AWS Cloud Development Kit (CDK)** — Use [AWS Cloud Development Kit \(CDK\)](#) (AWS CDK) as a software development framework for defining cloud infrastructure in code and provisioning it through AWS CloudFormation. Use the AWS CDK to define your cloud resources in a familiar programming language.

Establish scalable patterns to access approved public libraries - by creating separate kernels for common ML frameworks. Examples of such common ML frameworks include: Tensorflow, PyTorch, Scikit-learn, and Keras. This allows using internal repositories to give access to public libraries and create separate kernels for common ML frameworks.

Implementation plan

- **Use container technology** — Use [AWS Deep Learning Containers](#) or alternatively bring custom containers and store them in [Amazon Elastic Container Registry](#) (Amazon ECR). Using containers, you can train machine learning algorithms and deploy models quickly and reliably at any scale.
- **Use artifact repository** — Set up [AWS CodeArtifact](#) to be used as a central internal artifact repository.

This will enable pulling artifacts from internal repositories and reusing them.

Enable CI/CD/CT automation with traceability - Enable source code, data, and artifact version control of ML workloads to enable roll back to a specific version. Incorporate continuous integration (CI), continuous delivery (CD), and continuous training (CT) practices to ML workload operations. This will enable automation with added traceability.

- **Use Amazon SageMaker Pipelines** — Manual changes to a system can cost additional time and impair reproducibility. Changes to ML workload should be conducted, tracked and rolled back automatically. [MLOps](#) is a collection of best practices around integrating and deploying reproducible, auditable changes. MLOps increases your productivity while automating all facets of your ML development cycle (MLDC). [Amazon SageMaker Pipelines](#) is the first purpose-built, continuous integration (CI), continuous delivery (CD), and continuous training (CT) service. With SageMaker Pipelines, create, automate, and manage end-to-end ML workflows at scale.

Use an appropriate deployment and testing strategy - Run a tradeoff analysis across available and relevant deployment/testing strategies and select the one that meets your business requirements. Examples of deployment/testing strategies include blue/green, canary, shadow, and A/B testing. Implement metrics that evaluate model performance to identify when a

rollback or rollforward is required. When architecting for rollback or rollforward, evaluate the following for each model:

- Where is the model artifact stored?
- Are model artifacts versioned?
- What changes are included in each version?
- For a deployed endpoint, what version of the model is deployed?
- **Blue/green deployments** make use of a primary and secondary set of production environment. The blue environment is the existing infrastructure. New changes are deployed to a brand-new infrastructure stack, called the green stack. Once the deployment is complete, traffic is redirected from the blue stack to the green stack. For more details, review the [Blue/Green Deployments on AWS](#) whitepaper.
- In a **canary deployment**, new changes are rolled out in a limited fashion to the existing production environment. The purpose of a canary deployment is to reduce the risk of deploying a new version that impacts the workload. The method incrementally deploys the new version, making it visible to new users in a slow fashion. As you gain confidence in the deployment, you can deploy it to replace the current version in its entirety. For more details, review [performing a canary-based deployment](#).
- **A/B testing** is when you deploy a newer model to an endpoint alongside an existing model. You split the traffic between these two versions for an evaluation period. You then measure the performance of the new model against the existing model and, if it performs as expected, you then fully deploy the new model gradually by increasing traffic to it. For more details, review [test models in production](#) in the SageMaker documentation.
- A **shadow deployment** is where you release a new software version alongside an existing software version, then shift the traffic to the new version without impacting your production traffic. You can then evaluate the new version with production data without impacting your users.

Establish an automated re-training framework - Monitor the data and the model predictions. Run analyses of model performance against defined metrics to identify errors due to data and concept drift. Automate model re-training to mitigate these errors on fixed scheduled intervals, or when model variance reaches a defined threshold. Automated model retraining can also be started as enough new data becomes available.

- **Identify retraining opportunities** — Monitor data statistics and ML inferences at production using Amazon SageMaker Model Monitor. If the data drifts beyond a defined threshold, then start retraining. A retraining can also be initiated at defined scheduled intervals to meet business requirements. A retraining can also be started when additional training data is available. AWS supports mechanisms for automatically starting retraining based on new data PUT to an [Amazon S3](#) bucket. Ensure model versioning is supported when incorporating additional data into your models. This enables re-creating an inadvertently deleted model artifact using the combined versions of components used to create the versioned artifact.

- **Use Amazon SageMaker Pipelines** — A retraining pipeline can be developed using [Amazon SageMaker Pipelines](#) that enables orchestration using step creation and management.
- **Use AWS Step Functions** — You can also use [AWS Step Functions Data Science SDK for Amazon SageMaker](#) to automate training of a machine learning model. Define all the steps in the workflow and set up alerts to start the flow. To detect the presence of new training data in an S3 bucket, [AWS CloudTrail](#) combined with [Amazon CloudWatch](#) Events allows you to start an AWS Step Function workflow to initiate retraining tasks in your training pipeline.
- **Use third-party tools** — Use third-party deployment orchestration tools, such as [Jenkins](#), that integrate with AWS service APIs to automate model retraining when new data is available.

Ensure a recoverable endpoint with a managed version control strategy - Ensure an endpoint responsible for hosting model predictions, and all components responsible for generating that endpoint are fully recoverable. Some of these components include model artifacts, container images, and endpoint configurations. Ensure all required components are version controlled, and traceable in a lineage tracker system.

- **Implement MLOps best practices with Amazon SageMaker Pipelines and Projects** — [Amazon SageMaker Pipelines](#) is a service for building machine learning pipelines. It automates developing, training, and deploying models in a versioned, predictable manner. [Amazon SageMaker Projects](#) enable teams of data scientists and developers to collaborate on machine learning business problems. A SageMaker project is an [AWS Service Catalog](#) provisioned product that enables you to easily create an end-to-end ML solution. SageMaker Projects entities include pipeline executions, registered models, endpoints, datasets, and code repositories.
- **Use infrastructure as code (IaC) tools** — Use [AWS CloudFormation](#) to define and build your infrastructure, including your model endpoints. Store your CloudFormation code in git repositories, such as [AWS CodeCommit](#), so that you can version control your infrastructure code.
- **Use Amazon Elastic Container Registry (Amazon ECR)** — Store your containers in [Amazon ECR](#), an artifact repository for Docker containers. Amazon ECR automatically creates a version hash for your containers as you update them allowing you to roll back to previous versions.

Secure governed ML environment - Protect ML operations environments using managed services with best practices including: Detective and preventive guardrails, monitoring, security and incident management. Explore data in a managed and secure notebook environment. Centrally manage the configuration of Jupyter environments and enable self-service provisioning for the users.

Implementation plan

- **Break out ML workloads** by organizational unit access patterns. This will enable delegating required access to each group, such as administrators or data analysts.

- **Use guard rails and service control policies (SCPs)** to enforce best practices for each environment type. Limit infrastructure access to administrators.
- **Verify all sensitive data has access through restricted, isolated environments.** Ensure this by examining network isolation, dedicated resources, and service dependencies.
- **Secure ML algorithm implementation** using a restricted [Amazon SageMaker Jupyter](#) notebook environment. Secure model training and hosting containers by following the security processes required for your organization.

Evaluate machine learning deployment option (cloud versus edge) - Evaluate if machine learning applications require near-instantaneous inference results. Offering the lowest latency possible might require the removal of costly roundtrips to the nearest API endpoints. A reduction in latency can be achieved by running the inference directly on the device itself (on the *edge*). A common use-case for such a requirement is predictive maintenance in factories.

- **Evaluate cloud or device deployment options** — Training and optimizing machine learning models require massive computing resources, so it is a natural fit for the cloud. Inference takes a lot less computing power and is often done in real time when new data is available. When getting inference results with very low latency, ensure that your IoT applications can respond quickly to local events. Evaluate and choose the option to meet your business requirements.
- [Amazon SageMaker Neo](#) enables ML models to be trained once and then run anywhere in the cloud and at the edge. Amazon SageMaker Neo consists of a compiler and a runtime. The compilation API reads models exported from various frameworks, converts them into framework-agnostic representations, and generates optimized binary code. The runtime for each target platform then loads and runs the compiled model.
- [AWS IoT Greengrass](#) enables ML inferences on edge devices using models trained in the cloud. These models can be built using [Amazon SageMaker](#), [AWS Deep Learning AMI](#), or [AWS Deep Learning Containers](#). These models can be stored in [Amazon S3](#) before deployed on edge devices.

Use distributed training - Enable distributed training for a faster training time, when an algorithm allows it. Use multiple instances in a training cluster. Use managed services to help ensure all training instances are automatically shut down when training is completed.

Use Amazon SageMaker — The [distributed data parallel library](#) in Amazon SageMaker (the library) can be used to training large deep learning models that were previously difficult to train due to GPU memory limitations. The library automatically and efficiently splits a model across multiple GPUs and instances. It coordinates model training, allowing you to increase prediction accuracy by creating larger models with more parameters.

- The library extends the SageMaker training capabilities on deep learning models with near-linear scaling efficiency. This enables achieving fast time-to-train with minimal code changes.
- The library optimizes your training job for AWS network and Amazon EC2 instance topology.

- The library takes advantage of gradient updates to communicate between nodes with a custom allreduce algorithm.
- Increasing deep learning model size (layers and parameters) can result in better accuracy, however there is a limit to the maximum model size fitting a single GPU. When training deep learning models, GPU memory limitations can be a bottleneck in the following ways:
 - You can limit the size of the model you train. Given that larger models tend to achieve higher accuracy, this directly translates to trained model accuracy.
 - You can limit the batch size you train with, leading to lower GPU utilization and slower training.
 - To overcome the limitations associated with training a model on a single GPU, you can use model parallelism to distribute and train your model on multiple computing devices.

Secure inter-node cluster communications - For frameworks such as Tensorflow, it's common to share information like coefficients as part of the inter-node cluster communications. The algorithms require that exchanged information stay synchronized across nodes. Secure this information through encryption in transit.

Implementation plan

- **Enable inter-node encryption in SageMaker** — Distributed computing environments can be used for training, testing, and analysis. Some data can be transmitted between nodes that traverse wide networks, or even the Internet. Enable inter-node encryption through the appropriate controls for the technology choices made. You can instruct SageMaker to automatically [encrypt inter-container communication](#) for your training job. The data that passed between nodes is then passed over an encrypted tunnel without requiring any data encrypting and decrypting.
- **Enable encryption in transit in Amazon EMR** — There are many applications and execution engines in the Hadoop ecosystem, providing a variety of tools to match the needs of your ML and analytics workloads. [Amazon EMR](#) has distributed cluster capabilities and is also an option for running training jobs on the data that is either stored locally on the cluster or in [Amazon S3](#). Amazon EMR makes it easy to create and manage fully configured, elastic clusters of Amazon EC2 instances running Hadoop and other applications in the Hadoop ecosystem. Amazon EMR provides [security configurations](#) to set up data encryption at rest while stored on Amazon S3 and local [Amazon EBS](#) volumes. It also allows the set up of Transport Layer Security (TLS) certificates for the encryption of data in transit.

Monitor human interactions with data for anomalous activity - Ensure that data access logging is enabled. Audit for anomalous data access events, such as access events from abnormal locations, or activity exceeding the baseline for that entity. Use services and tools that support anomalous activity alerting, and combine their use with data classification to assess risk. Evaluate using services to aid in monitoring data access events.

- **Enable data access logging** — Verify that you have data access logging for all human CRUD (create, read, update, and delete) operations, including the details of who accessed what elements, what action they took, and at what time.
- **Classify your data** — Use [Amazon Macie](#) for protecting and classifying training and inference data in [Amazon S3](#). Amazon Macie is a fully managed security service. It uses ML to automatically discover, classify, and protect sensitive data in AWS. The service recognizes sensitive data, such as personally identifiable information (PII) or intellectual property. Amazon Macie provides visibility into how this data is being accessed or moved. Amazon Macie continually monitors data access activity for anomalies. It generates detailed alerts when it detects a risk of unauthorized access or inadvertent data leaks.
- **Monitor and protect** — Use [Amazon GuardDuty](#) to monitor for malicious and unauthorized activities. This will enable protecting AWS accounts, workloads, and data stored in [Amazon S3](#).

Monitor usage and cost by ML activity - Use cloud resource tagging to manage, identify, organize, search for, and filter resources. Tags help categorize resources by purpose, owner, environment, or other criteria. Associate costs with resources using ML activity categories, such as re-training and hosting, by using tagging to manage and optimize cost in deployment phases. Tagging can be useful for generating billing reports with breakdown of cost by associated resources.

- **Use AWS tagging** — A [tag](#) is a label that you or AWS assigns to an AWS resource. Each tag consists of a key and a value. For each resource, each tag key must be unique, and each tag key can have only one value. You can use tags to organize your resources, and cost allocation tags to track your AWS costs on a detailed level. AWS uses the cost allocation tags to organize your resource costs on your cost allocation report. This will make it easier for you to categorize and track your AWS costs. AWS provides two types of cost allocation tags, an AWS-generated tag and user-defined tags.

Stop resources when not in use to reduce cost. For example, hosted Jupyter environments used to explore small samples of data, can be stopped when not actively in use. Where practical, commit the work, stop them, and restart when needed. The same approach can be used to stop the computing and the data storage services.

- **Use CloudWatch Billing Alarms** — You can monitor your estimated AWS charges by using [Amazon CloudWatch](#). When you enable the monitoring of estimated charges for your AWS account, the estimated charges are calculated and sent several times daily to CloudWatch as metric data. Use this feature to receive notifications when your resource charge exceeds a threshold amount.
- **Use SageMaker Lifecycle Configuration** — A [lifecycle configuration](#) provides shell scripts that run when you create the notebook instance or whenever you start one. When you create a notebook instance, you can create a new lifecycle configuration and its scripts or apply ones that you already have. Use a lifecycle configuration script to access AWS services from your

notebook. These scripts enable checking notebook instance activities and shut them down if idle.

- **Use Amazon SageMaker Studio auto shutdown** — [Amazon SageMaker Studio](#) provides a unified, web-based visual interface for performing all ML development steps, making data science teams more productive. Idle SageMaker Studio notebooks can be detected and stopped using an auto-shutdown JupyterLab extension that can be [installed manually or automatically](#). You can shut down individual resources, including notebooks, terminals, kernels, applications, and instances.

MLCC TACTICAL USE CASE #16: ML INFERENCE DELIVERY

Resolution Life would like to design and implement an optimized workflow to operationalize the set of ML models. Typical Inference workflow consists of – Data producers, ML model artifacts and model output consumers. Data producers may include a disparate set of data sources, from which model specific features will be collected/ingested, required features engineered (transformed) and consolidated before making it available for the model inference.

ML Model artifacts include machine learning model file (binaries) and supporting artifacts. Different considerations in this aspect could be inference time, real/batch execution, hosting environment, model retraining support and ease of maintenance. Model consumers may include online/offline dashboards and reports or other models in the workflow.

Resolution Life would like to develop an ML DevOps framework that leverages easily accessible templates for developing and promoting ML model artifacts, reducing overall deployment times. Such an automated ML DevOps framework enables Resolution Life data teams to utilise deployment patterns based on infrastructure templates and architectural blueprints. Templates are usable by any data team in Resolution Life and are incorporated into automatic approval and sign-off processes. Using pre-approved templates gives teams the opportunity to operate autonomously within centrally enforced controls and boundaries, without having to go through redundant approval processes. Workload teams are therefore indirectly incentivised to spin-up such templated, pre-approved development infrastructure, as it enables them to deploy quicker. Templates can also incorporate coding best practices, for example on naming conventions, modularization, or tests, as code quality influences time and efforts needed for code approvals.

Existing Resolution Life processes ensuring optimal workload placement and quality assurance, might have to be altered, but can be utilised within the MLDevOps framework. The respective operational procedures are adjusted and simplified to leverage these automation capabilities. Implementation teams will have to templatised their typical workload patterns and cooperate with the Resolution Life CISO organisation to initially bake in security regulations into the templates (e.g. via pre-approved containers). To cover best practices around cost control and

cost transparency in the cloud, alternative paths to production can be simulated for every ML model, hence defining the optimal setup with respect to cost and latency estimations. Such simulation capabilities can be an efficient and autonomous way for teams to right-size training and model hosting instances or account for inference architecture based on consumption patterns. As templates can be added over time, they are managed in a library fashion. New templates will come from Resolution Life's own successful ML products, displaying new consumptions patterns, additional ML model architectures, or entirely new paths to production.

CHALLENGES

It is understood that Resolution Life faces the following challenges in delivering this area today:

- **Data availability:** Each ML Model will have a specific data-need both in terms of attributes and pre-processing required. The workflow needs to make sure required data and with desired state is available for the model on time.
- **Optimized workflow orchestration:** Given there are 100+ jobs running AS-IS with the need to increase the scope further in future, the workflow needs to be optimized in terms of cost and compute. Allowing resources to be provision on need basis alone with minimal latency, while achieving this in serverless fashion with auto scale in/out capability.
- **Fully Microservice based architecture:** The core aspect for efficient model management and maintenance lies in decoupling the individual jobs to allow fully automated and isolated updates and usage. All while, provisioning enough resources dynamically. For e.g., currently, the containers are looked at as job containers and all the code and libraries today exist in S3 buckets. The best practices are to bake the containers early and promote them through the environments. This reduces overall maintenance efforts and forces us to fail fast and early

OBJECTIVES

The overall goal of this initiative is to provide structures, processes, and policies for implementing an efficient model Inference Delivery framework that would help:

- Speed-up deployments by enabling CI/CD blueprints for numerous teams and ML use cases within Resolution Life;
- Reduce time to create paths to production for use cases by automating template execution with a few clicks, having the respective access controls built-in;
- Have security and governance controls baked into deployment templates to stay within regulatory constraints without causing delays in the productionisation of ML models;
- Automate the process of finding optimal deployments in the cloud with respect to latency and cost;
- Create an enterprise-wide ML Inference platform
- Enable ease of tracking and automated updates across different jobs

- Provide the framework with minimal cost and maintenance need
- Create role-based access to the Inference platform, for
 - Data Science
 - Machine Learning Engineers
 - Data Engineers
 - Data Owners
 - DevOps Developers

MEASUREMENT METRICS

Measurement metrics could include the following:

- **Data Availability Latency:** The time required to fetch, ingest, pre-process and consolidate the data required for model inference.
- **Model Inference Latency:** The time taken by model for inference. Latency requirement of specific model will be part of the requirement and will depend on different factors.
- **Need for manual interference:** Ideal inference workflow will be fully automated with provision for manual approval in key steps

AWS SERVICE IMPLEMENTATION NOTES

The following elements from the AWS Well Architected Framework ML Lens will be relevant.

[Use APIs to abstract change from model consuming applications](#) - Use a flexible application and API design to abstract change from model consuming applications. Ensure that changes to an ML model are introduced with minimal or no interruption to existing workload capabilities. Minimize the changes across other downstream applications.

- **Adopt best practices in use of APIs** — Expose your ML endpoints through APIs so that changes to the model can be introduced without disrupting upstream communications. Document your API in a central repository or documentation site so that any calling services can easily understand your API routes and flags. Ensure that any changes to your API are communicated with any calling services.
- **Deploy a model in Amazon SageMaker** — After you train your model, you can deploy it using [Amazon SageMaker](#) to get predictions. To establish a persistent endpoint to get one prediction at a time, use SageMaker hosting services. To get predictions for an entire dataset, use SageMaker batch transform.
- **Use Amazon API Gateway to create APIs** — [Amazon API Gateway](#) is a fully managed service that enables developers to create, publish, maintain, monitor, and secure APIs. Using API Gateway, you can create RESTful APIs and WebSocket APIs that enable real-time two-way communication applications. API Gateway supports containerized and serverless workloads, as well as web applications.

[Adopt a machine learning microservice strategy](#) - To ensure changes to an ML model are introduced with minimal or no interruption to existing workload, adopt a microservice instead

of a monolithic architecture. This approach replaces one large resource with multiple small resources and can reduce the impact of a single failure on the overall workload. This strategy enables distributed development and improves scalability, enabling easier change management.

- **Adopt a microservice strategy** — Service-oriented architecture (SOA) is the practice of making software components reusable using service interfaces. Instead of building a monolithic application, where all functionality is contained in a single runtime, the application is instead broken into separate components. Microservices extend this by making components that are single-purpose and reusable.
- **When building your architecture, divide components** along business boundaries or logical domains. Adopt a philosophy that favors single-purpose applications that can be composed in different ways to deliver different end-user experiences.
- **Use AWS services in developing microservices** — Two popular approaches for developing microservices are using [AWS Lambda](#) and Docker containers with [AWS Fargate](#). With AWS Lambda, you can run code for virtually any type of application or backend service with zero administration. You pay only for the compute time you consume, and there is no charge when your code is not running. A common approach to reduce operational efforts for deployment is using a container based deployment. AWS Fargate is a container management service that allows you to run serverless containers so you don't have to worry about provisioning, configuring, and scaling clusters of virtual machines to run containers.

[Allow automatic scaling of the model endpoint](#) - Implement capabilities that allow the automatic scaling of model endpoints. This helps ensure the reliable processing of predictions to meet changing workload demands. Include monitoring on endpoints to identify a threshold that initiates the addition or removal of resources to support current demand. After a request to scale is received, put in place a solution to scale backend resources supporting that endpoint.

- **Configure automatic scaling for Amazon SageMaker Endpoints** — [Amazon SageMaker supports automatic scaling \(autoscaling\)](#) for your hosted models. SageMaker Endpoints can be configured with autoscaling. This ensures that as traffic increases in your application your endpoint can maintain the same level of service availability. Automatic scaling is a key feature of the cloud. It allows you to automatically provision new resources horizontally to handle increased user demand or system load. Automatic scaling is also a key component of creating event-driven architectures and is a necessary capability of any distributed system.
- **Use Amazon Elastic Inference** — With [Amazon Elastic Inference](#), you can choose the CPU instance in AWS that is best suited to the overall compute and memory needs of your application. Separately configure the right amount of GPU-powered inference acceleration, allowing you to efficiently utilize resources and reduce costs.
- **Use Amazon Elastic Inference with EC2 Auto Scaling** — When you create an Auto Scaling group, you can specify the information required to configure the [Amazon EC2](#) instances. This includes Elastic Inference accelerators. To do this, specify a launch template with your instance configuration and the Elastic Inference accelerator.

Optimize training and inference instance types - Determine how the model type and data velocity affect the choice of training and inference instance types. Identify the right instance type that supports memory intensive training, or compute intensive with high throughput and low latency real-time inference. The speed of model inferences is directly impacted by model complexity. Selection of high compute instances can accelerate inference speed. GPUs are often the preferred processor type to train many deep learning models. CPUs are often sufficient for the inference workloads.

- **Experiment with alternative instance types to train and deploy** — Use [Amazon SageMaker Experiments](#) to organize, track, compare, and evaluate your machine learning experiments. Learn which instance types are most appropriate for the ML algorithm and the use case. In [Amazon SageMaker](#), start with the default or example instance types, and then scale to optimize performance and cost. Use multiple instances for training for large datasets to take advantage of scale.

Establish deployment environment metrics - Measure machine learning operations metrics to determine the performance of a deployed environment. These metrics include memory and CPU/GPU usage, disk utilization, ML endpoint invocations, and latency.

- **Record performance-related metrics** — Use a monitoring and observability service to record performance-related metrics. These metrics can include database transactions, slow queries, I/O latency, HTTP request throughput, service latency, and other key data.
- **Analyze metrics when events or incidents occur** — Use monitoring dashboards and reports to understand and diagnose the impact of an event or incident. These views provide insight into what portions of the workload are not performing as expected.
- **Establish key performance indicators (KPIs) to measure workload performance** — Identify the KPIs that indicate whether the workload is performing as intended. An API-based workload might use overall response latency as an indication of overall performance. For example, an ecommerce site might choose to use the number of purchases as its KPI.
- **Use monitoring to generate alarm-based notifications** — Monitor metrics for the defined key performance indicators (KPIs) and generate alarms automatically when the measurements are outside expected boundaries.
- **Review metrics at regular intervals** — As routine maintenance, or in response to events or incidents, review what metrics are collected. Use these reviews to identify the metrics that were key in addressing issues. Identify any additional tracked metrics that would help to identify, address, or prevent issues.
- **Monitor and alarm proactively** — Use key performance indicators (KPIs), combined with monitoring and alerting systems, to proactively address performance-related issues. Use alarms to initiate automated actions to remediate issues where possible. Escalate the alarm to those able to respond if an automated response is not possible. Use a system to predict expected KPI values, and alert when thresholds are exceeded. Use a tool that can automatically halt or roll back deployments if KPIs are outside of the expected values.
- **Use Amazon CloudWatch** — Use [Amazon CloudWatch](#) metrics for SageMaker to determine the memory, CPU usage, and disk utilization. Set up [CloudWatch Dashboards](#) to

visualize the environment metrics and establish [CloudWatch alarms](#) to initiate a notification via [Amazon SNS](#) (Email, SMS, WebHook) to notify on events occurring in the execution environment.

- **Use Amazon EventBridge** — Consider defining an automated workflow using [Amazon EventBridge](#) to respond automatically to events. These events can include training job status changes, endpoint status changes, and increasing the compute environment capacity after it crosses a defined threshold (such as CPU or disk utilization).
- **Use AWS Application Cost Profiler** — Use [AWS Application Cost Profiler](#) to report the cost per tenant (model/user).

Protect against adversarial and malicious activities - Add protection inside and outside of the deployed code to detect malicious inputs that might result in incorrect predictions.

Automatically detect unauthorized changes by examining the inputs in detail. Repair and validate the inputs before they are added back to the pool.

- **Evaluate the robustness of the algorithm** — Evaluate your use case and determine bad predictions or classifications. Evaluate the robustness of the algorithm against increasingly perturbed inputs to understand susceptibility to manipulated inputs.
- **Build for robustness from the start** — Select diverse features to improve the algorithm's ability to handle outliers. Consider pairing models in an ensemble for increased diversity in decisions. Consider using ensembles to build in robustness around decision points.
- **Identify repeats** — Detect similar repeated inputs to the model to indicate possible threats to the decision boundaries. This can take the form of model brute forcing, where threats iterate only a limited set of variables to determine what influences decision points and derive feature importance.
- **Lineage tracking** — If retraining on untrusted or invalidated inputs, make sure any model skew is traced back to the data and pruned before retraining a replacement model.
- **Use secure inference API endpoints** — Host the model so that a consumer of the model can perform inference against it securely. Enable consumers using the API to define the relationship, restrict access to the base model, and provide monitoring of model interactions.

PRIORITIZATION

The purpose of the Solutioning phase is to dive deep into the technical details, success metrics, and plan to address the selected target (MVP, etc.). Each has been weighted on four metrics:

1. **Data Availability:** This is the general ability to access and process data required to realize the building block project. A higher score indicates a higher availability of data upon project inception. In cases where data already resides in AWS (e.g., S3, etc.), a higher score will be also granted as less up-front work has to be done in terms of data curation, data privacy assessments, and data transfer.
 - Breadth and depth of internal data
 - Breadth and depth of external data

- Annotated internal and external data
 - Data available on AWS
 - Data availability from client side e.g. apps and browsers
2. **Ability to Execute:** This is the ability to use the outputs of the building block project directly in the business or operations. Projects which require less effort (e.g., where efforts may include additional development time, process transformation, etc.) to use these outputs will receive a higher score. In addition, if a significant amount of training is expected, organizational resistance is expected, or predecessors exist, a lower score will be assessed.
 - Ability to measure
 - Capability to deploy exists
 - Management processes in place
 - Process can be scaled / extended for production
 3. **Business Value:** This is the intrinsic value to the business, which can be measured by revenue improvement, cost reduction, improved customer experience, or other similar factors. A higher score indicates a higher business value.
 - A Pro Forma ROI will be created to show the value of the building block projects when delivered together

RECOMMENDED NEXT STEPS

In order to realize the value of the aforementioned building block projects, the following next steps are recommended to Resolution Life:

1. PHASE 1 RECOMMENDED ROADMAP:
 - a. ML Data Pipeline
 - i. Data Ingest Pipeline
 - ii. ML Data Engine
 - b. ML Model Development Engine
 - i. ML Experimentation
 - ii. ML Model Management
 - c. ML DevOps Framework
 - i. Evaluation and Performance
 - ii. ML Implementation Pipeline
 - iii. ML Inference Delivery
 - d. Execute and monitor the progression of the target states
 - i. Refine the plan to support Resolution Life's long-term ML Competency Center design and deployment
2. Parallel Next Steps:
 - a. Align the Resolution Life resources to work with the AWS MLCC team
 - b. Allocate a sandbox environment to validate the infrastructure
 - c. Document and address permissions, tool provisioning, and regulatory needs

d. Schedule Kickoff

OPTIONS FOR SUPPORT

For the building block projects fully assessed in this document, Resolution Life may pursue many paths, either on their own, with limited assistance from outside parties, or with significant assistance from outside parties. With data scientists on staff, and several others in supporting roles (e.g., data engineering,), Resolution Life has the option of any of these paths. Resolution Life has indicated that they are initially looking for assistance with the four high priority projects. As such, AWS has two discrete ways to assist Resolution Life in supporting their Machine Learning (ML) use cases and programs:

- **AWS ProServe:** This team has experts in ML, as well as in big data, ingestion and processing, architecture, and full stack deployment. This team provides skill augmentation for MVPs and project delivery, custom development and modeling, application and services integration, and hands-on technical workshops. This is most recommended option for Resolution Life. In addition, there are ML specialists provide advisory services to our customers. They can help in several areas, such as brainstorming ML solutions, data preparation, custom ML modeling, application services integration to make results actionable, and educational programs, such as technical boot camps.
- **AWS Partner Network:** This is set of ML specialist partners who deliver and support ML solutions. They also provide advisory services, ML solution architecture and development, and custom modeling and services integration.

ADDITIONAL NOTES & ASSUMPTIONS

The following additional notes and assumptions have been made in assessing Resolution Life's needs:

- The Customer will provide anonymized sample data for use in a non-production environment via Amazon Simple Storage Service (Amazon S3) bucket(s)
- Any data provided must be in the following format(s): JSON, CSV, Parquet, Avro, TXT (text data), JPEG or PNG (image data), MPEG-4 or MOV (video data), 16kHz or 8kHz audio streams (audio data)
- If data pre-processing is required, there must be one (1) week or less of data processing work involved based on AWS's data scientist's estimate. If the estimate is longer than one (1) week, Customer is required to apply necessary data processing in advance of the engagement

- If models are developed, Customer will ensure that the data provided contains sufficient relevant data to inform meaningful predictions both in the form of rich features and historical data
- If data is provided, Customer will provide AWS with a description of relevant data sources and attributes
- If models need to be developed Customer will provide necessary subject matter expertise to assist in model training, validation, testing, and deployment.
- Customer will make resources available to participate in the project throughout the project timeline
- Customer resources will be available to travel for training

For Internal Use Only

PARTICIPANTS

Resolution Life	AWS
Mittal Bhiogade	Raj Kumar
Mohan Gurjar	Zorina Alliata
	James Poquiz
	Steve Bakanov

For Internal Use Only